華中科技大學

本科生毕业设计

面向浪潮 ZNS SSD 设备的写入瓶颈分析与性能优化

院　　系	计算机科学与技术学院
专业班级	计科本硕博 2001 班
姓　　名	柳子淇
学　　号	U202015628
指导教师	刘渝

2024 年 5 月 29 日

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：柳子淇 2024 年 5 月 29 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保　密□，在　　年解密后适用本授权书

2、不保密☑。

（请在以上相应方框内打“√”）

作者签名 柳子淇 2024 年 5 月 29 日

导师签名：刘伶 2024 年 5 月 29 日

摘 要

分区命名空间（Zoned Namespace，ZNS）固态硬盘通过区域化管理并强制要求区域（Zone）内顺序写入，有效解决了传统 SSD 的写放大和垃圾回收开销过高等问题。相较于传统 SSD，ZNS SSD 拥有更高的吞吐量，更低更稳定的延迟和更长的寿命。然而，现有工作大多基于模拟器实现，得到的结论可能与实践不符；另一方面，现有适配工作未能充分发挥 ZNS 的性能。

本文基于对 ZNS 模拟器和真实 ZNS 设备的对比测试与评估，结合对日志式合并树存储模型和对 RocksDB 存储引擎的深入剖析，提出一种区域追加感知型写前日志 A2-WAL（Append-Aware Write-Ahead Log），并实现了 ZAFS（Zone-Append File System），一个基于 SPDK 的 ZNS 专用文件系统。通过引入 SPDK 技术，利用区域追加命令和 A2-WAL 的异步写入机制，能够有效解决 RocksDB 在高写入场景下由于 WAL 串行写入导致的性能瓶颈问题。

通过实验，获得了 ZNS SSD 硬件测试数据，并基于此探讨了 ZNS SSD 的设计理念和参数选择，对 ZAFS 的有效性进行了评估和验证。结果表明，相比于 ZenFS，ZAFS 使吞吐量提升了 1.9~3.5 倍。此外，改进虽然增加了 WAL 文件大小，但在灾害恢复时，数据恢复时间最多能够降低 6.7 倍。

关键词：分区命名空间固态硬盘；日志式合并树；写前日志；键值存储；文件系统

Abstract

The Zoned Namespace (ZNS) solid-state drive (SSD) resolves the write amplification and garbage collection overhead issues inherent in traditional SSDs by managing data through zoned namespace and enforcing sequential writes within each zone. Compared to traditional SSDs, ZNS SSDs exhibit higher throughput, lower and more stable latency, and longer lifetimes. However, most existing works are based on simulator implementations, which may not reflect real-world scenarios; furthermore, adapting applications to utilize ZNS's hardware features is essential, but existing adaptations fail to fully exploit ZNS's performance.

This paper presents a comparative evaluation of ZNS simulators and real ZNS devices, combined with an in-depth analysis of log-structured merge trees and the RocksDB storage engine. We propose an append-aware write-ahead log (A2-WAL) that takes into account zone appending, and implement ZAFS (Zone-Append File System), a ZNS-specific file system based on SPDK. By leveraging SPDK technology, we utilize zone appending commands and A2-WAL's asynchronous writing mechanism to effectively address the performance bottleneck issue caused by RocksDB's serial write-ahead logging in high-write scenarios.

Through experiments, we obtain ZNS SSD hardware test data and explore the design principles and parameter selection of ZNS SSDs, evaluating and validating the effectiveness of ZAFS. The results show that, compared to ZenFS, ZAFS achieves a throughput improvement of 1.9-3.5 times. Moreover, although A2-WAL increases file size, the data recovery time can be reduced by up to 6.7 times in disaster recovery scenarios.

Keywords: Zoned Namespace SSD, Log-Structured Merge Tree, Write-Ahead Log, Key-Value Storage, File System

目 录

摘 要 ..... **I**
Abstract ..... **II**
1 绪 论 **1**
1.1 课题背景、目的与意义 1
1.2 国内外研究现状 5
1.3 论文的主要内容与结构 8
2 相关技术基础 **10**
2.1 几种新兴 SSD 技术 10
2.2 键值存储与 LSM-Tree 模型 11
2.3 Linux I/O 14
2.4 本章小结 18
3 ZAFS 的设计与实现 **19**
3.1 ZNS 的基本环境 19
3.2 现有方案的局限性与问题分析 24
3.3 ZAFS 架构概览 33
3.4 追加感知型 WAL 34
3.5 ZAFS 的实现 37
3.6 本章小结 41
4 性能测试与分析 **43**
4.1 实验平台配置 43
4.2 模拟器与真实设备的对比评估 43
4.3 基于 Inspur NS8600G2-Z 的 ZNS SSD 评估 45
4.4 ZAFS 评估 49
4.5 本章小结 50
5 总结与展望 **51**
致 谢 **52**
参考文献 **53**

1 绪 论

本章首先分析传统固态硬盘设计上存在的局限和发展中遇到的瓶颈，然后介绍新兴的固态硬盘技术及其国内外研究发展现状，深入分析其优势和存在的风险与挑战，并对本文的主要研究内容及工作意义作了具体说明。

1.1 课题背景、目的与意义

随着数据中心对存储性能与效率的需求日益增长，传统的固态硬盘在性能，单盘容量，寿命等方面已经无法满足其存储需求。在此背景下，Zoned Namespace (ZNS) SSD 作为一种新兴的固态盘存储技术，通过区域化管理并强制要求区域（Zone）内顺序写入，消除了硬盘内部管理机制造成的写放大效应，有效应对了挑战，显著提升了存储效率，延长了固态硬盘寿命。获得学术界和工业界的青睐。本课题来源于实验室与浪潮等企业联合申请的“面向新型计算模式的分布式存储系统”重点专项，尝试利用 ZNS SSD 等新型硬件重新设计存储系统。本课题作为此专项的先行性研究，基于多块实际的 ZNS SSD 设备实现了 ZNS SSD 的数据访问特征分析以及性能优化。

1.1.1 研究背景与趋势

传统固态硬盘在设计上存在较大的改进空间。例如，针对闪存转换层和垃圾回收机制带来的写放大问题，兴起了在软件层面对固态硬盘进行优化的研究工作。然而，这些技术并不能彻底解决固态硬盘硬件设计上的缺陷。因此，工业界向底层深入，从硬件层面进行了改良和创新。但是，这些新硬件技术的发展仍然需要软件层面的适配性支持，以发挥其硬件优势，更好的适应实际应用的发展。

写放大倍数（Write Amplification Factor，WAF）是评价存储系统效率的关键指标之一，直接影响写入效率和设备寿命。I/O 路径中引发写放大的因素众多，包括应用程序的随机写入特性以及副本冗余设计问题；文件系统和操作系统的结构设计问题； FTL 层的垃圾回收机制问题，磨损均衡策略问题以及闪存颗粒擦写粒度不一致问题等等。针对这些问题，目前优化写放大的研究主要集中在以下几个方面：

（1）文件系统优化：通过改进日志结构文件系统（Log-structured File System，LFS[1]）的数据管理策略，利用追加写的方式减少数据的原地更新，从而降低垃圾回收的频率。例如 F2FS[2]、BetrFS[3]等，这些工作的核心点在于分离冷热数据，尽可能将冷热程度相同的数据块放置于同一区域，使其能在相近的时间内失效，从而降低写放大。

（2）SSD 数据放置策略优化： 研究如何在 SSD 内部有效区分冷热数据，将相同热度的数据存放在一起，以减少垃圾回收时的数据迁移，降低写放大。例如 Yang 等人[4]提出的一种通过预测数据块温度来放置数据的方案，其使用 K-Means 算法进行聚类，将温度相近的数据分发到相同的 NAND 块。ML-DT[5]根据 I/O 大小、逻辑地址、历史块寿命等特征生成预测模型，对数据块的未来寿命进行预测，从而实现冷热数据分离。

（3）垃圾回收算法优化：当 SSD 内部失效数据占比超过一定的阈值后，控制器会触发垃圾回收操作。现有的垃圾回收算法旨在根据不同的指标，在候选区域中选择最合适的部分进行回收开发更高效的垃圾回收算法，通过优化数据回收顺序和时机，减少写放大。例如 Windowed Greedy[6]，Random-Greedy[7]，d-choices[8]等。

然而，上述研究工作只能从操作系统、应用层面上缓解工作负载导致的写放大，不能解决固态硬盘内部的写放大现象。这一方面缘于闪存转换层（Flash Translation Layer, FTL）的管理策略，另一方面则是由于固态硬盘中 NAND 闪存颗粒擦写粒度不一致导致的。

FTL 是固态硬盘的核心组件之一，目的是向操作系统屏蔽固态硬盘的底层特性，使硬盘作为块设备使用。该转换层负责维护硬盘逻辑地址到物理地址的映射表，同时，垃圾回收，磨损均衡，资源调度，坏块管理等策略都实现在 FTL 层中。FTL 层的设计初衷是兼容传统块设备，但在存储架构日益复杂的发展下，暴露出新的问题。例如，在更新一个块的过程中，如果 FTL 给出的目标物理地址无法写入时，需要更改映射表，重新分配一个“新块”进行写入，然后对旧块进行回收，将其中未失效的内容迁移到“新块”。这个过程会造成写入延时和写放大。此外，所谓的“新块”可能是已经存有部分数据的地址空间，仍可能因为无法容纳

（或其他原因）被判断为无法写入，需要再分配一个“新块”，从而引发链式数据迁移，以不可预测的性能开销严重影响主机的工作。

除了性能开销外，FTL 也会带来额外的空间开销。FTL 用于维护细粒度(例如 4KB)的映射表需要占用的 DRAM 空间约为硬盘总空间的 1%。此外，盘内也存在闪存空间浪费。例如，为了提高随机写入的性能，SSD 采用预留空间（Over-Provisioning）策略，通过预留专用空间来保证垃圾回收等数据腾挪操作的效率。

此外，闪存介质的擦写特性也是导致写放大的因素。由于 NAND 闪存的擦除操作是以可擦除块为单位进行的，而擦除的大小通常远大于页面大小（即数据写入的最小单位），这就意味着即使是对单个页面的小范围修改，也可能导致整个块的擦除，迁移和重写，进而产生写放大。

针对这些问题，近期，一些新兴 SSD 技术正在兴起，并取得了一些进展。例如：Multi-Stream SSD [9]，Open-Channel SSD (OC SSD) [10]，KV SSD[11]等等。这些技术旨在突破传统 SSD 硬件设计的局限性，其中最新颖，最具有潜力的是分区命名空间固态硬盘——ZNS SSD[12]。

Zoned Namespace SSD（ZNS SSD）技术是为了应对传统 SSD 中存在的写放大问题而设计的一种新型固态硬盘。与传统 SSD 采用的基于细粒度随机映射的均匀空间管理策略不同，ZNS SSD 引入了分区（或"区域"）的概念，将闪存介质的地址空间划分为若干独立的、非重叠的区域，且严格限制每个区域内的写入操作必须是顺序的。这种设计增大了硬盘内部的空间管理粒度，显著减少了因闪存转换层导致的垃圾回收操作，消除了盘内的写放大现象。

ZNS SSD 通过区域化管理数据，具体包括定义区域容量、活动区域限制和区域追加命令等新属性，向主机暴露了更多的存储器内部信息，使 SSD 不再是“黑盒”设备。并且，ZNS 强制要求各个区域内只能通过 write 命令或者 append 命令顺序写入，避免了传统硬盘中 FTL 层的种种开销，无需使用昂贵的 DRAM 资源维护映射表。ZNS 直接消除了盘内写放大现象，提高了 SSD 性能和寿命。并且，经过 SSD 制造商、软件开发者和标准化机构之间的合作努力，ZNS SSD 技术已有稳定的硬件标准和基础软件库。

在标准化方面，NVMe 组织已定义了 Zone 接口语义和规范，并在 Linux 内

核中实现了 NVMe ZNS 支持。在软件生态方面，开源社区正在积极推动对 ZNS SSD 的支持，并集成到操作系统、数据库等存储系统中，以充分利用其优势。已有多种针对 ZNS 优化的文件系统和工具，帮助开发者和企业更好地理解和利用该技术。例如，像 F2FS 等为闪存设计的文件系统已经支持 ZNS，另外也有为数据库存储引擎适配 ZNS 而设计的文件系统 ZenFS，这些工作显示了 ZNS 技术的实际应用潜力。

综上，ZNS SSD 技术的优势如下：

（1）消除写放大，延长寿命：ZNS SSD 的摒弃了原有的妥协性设计，消除了 FTL 层，并使用区域作为管理的最小单元，这种数据管理方式与闪存介质的物理特性相契合，可以消除盘内写放大现象，延长闪存介质的寿命。

（2）更高的吞吐量，更低的延迟：ZNS SSD 通过将数据放置和垃圾回收操作交由主机处理，可以实现更优化的数据放置和更可预测的垃圾回收，从而提高性能。

（3）更低的成本：由于 ZNS SSD 不需要复杂的设备控制器算法，可以简化硬件设计，并且不需要过多的板载 DRAM，降低了成本。

（4）更高的容量利用率，使得超大容量 SSD 成为可能：ZNS SSD 使用更大的管理粒度，并且节省了 OP（Over-provision）空间，可以更有效地利用存储容量。另一方面，传统 SSD 的 FTL 层性能开销和 DRAM 空间需求会随容量激增，导致单盘容量过大时，带来不可接受的开销。而 ZNS 突破了二者的限制，使得成本和性能更加可控，因此，使用 ZNS 技术让超大容量 SSD 的出现成为可能。

1.1.2 面临的问题和挑战

ZNS SSD 技术的设计和实施虽然带来了显著的性能与成本优势，但也面临着若干严峻的问题，并对存储系统的设计和实际应用带来影响与挑战。

首先是技术适应性和兼容性问题，由于 ZNS SSD 采用了与传统 SSD 不同的数据管理策略，基于新的基于 Zone 语义实现了新的接口，这要求现有的操作系统和应用软件必须进行相应的调整以充分利用 ZNS SSD 的性能优势，尽管已经有一部分工作实现了对 ZNS 的支持，但充分发挥 ZNS 的特性仍需要更精巧的设计。这些问题可能导致在初期实施过程中为使用者带来兼容性成本和技术学习成

本。

其次，是主机端管理复杂性增加。ZNS SSD 将信息暴露给主机的同时增加了存储管理的复杂性。例如，操作系统或数据库需要主动地将数据分配到不同的区域，并管理每个区域的状态和性能，才能充分利用 ZNS 的特性。这就需要更精细的资源调度算法和更高级的系统监控工具，然而，目前适配 ZNS 的工程实现仍比较匮乏。

1.1.3 课题目的与意义

本课题旨在深入分析基于 Zoned Namespace (ZNS) SSD 的数据访问特征，并通过研究针对这些特性的性能优化策略，探索提升基于 ZNS SSD 的存储系统的性能的有效方法。一方面，相较于已有的在 QEMU[13]、FEMU[14]等模拟器上的工作，此项工作为研究 ZNS SSD 在真实场景下的性能表现提供理论和数据基础。另一方面，本课题优化了应用对 ZNS 的适配，能更充分地发挥其硬件特性，从而在实际应用中提升存储系统的效率和可靠性。此外，随着数据中心对存储解决方案效率和性能要求的不断提升，该方向的研究有望降低 SSD 使用成本，提升存储效率，推动 SSD 的大规模使用，满足未来数据中心更为复杂和高效的存储需求。

1.2 国内外研究现状

目前，关于 ZNS 已经有了一些初步的研究。主要是集中在底层设计，如硬件优化[13][14][15]，文件系统优化[16]等，或是在特定应用场景下的适配和优化[17][18]和探索性工作[20]。

在 ZNS 上的研究可以大致分为三个层级：在硬件方面，已有领先的 SSD 制造商，例如三星和西部数据，开始发布 ZNS 产品并展开研究，例如 ZNS+[13] 、eZNS[14]等；在系统方面，已经有支持 ZNS 的文件系统如：F2FS[15]等， 也有一些基于 Linux 系统的创新性工作，如利用将 ZNS 盘用作交换分区[16]。在相关应用方面，现有的基于 ZNS 的应用主要集中在键值存储领域，尤其是基于 LSM-tree 的键值存储优化，考虑到 ZNS SSD 和 LSM-tree 都遵循严格的顺序写约束，其软硬件特征高度重合， 将一些应用移植到 ZNS SSD 上会获得更好的性能。典

型案例如基于 ZenFS[12]的 RocksDB[17] 。下面简要介绍三篇突出体现 ZNS 硬件特性的相关论文及其核心思想，并进行分析。

1.2.1 ZNS+：设备内压缩

ZNS+是一种高级区域命名空间接口（Zoned Namespace Interface），它支持在存储设备内部进行区域压缩（In-Storage Zone Compaction）。ZNS+是为了解决日志文件系统（LFS）在执行段压缩时的高消耗问题而提出的。它允许主机将数据复制行为下放到 SSD 存储器层面异步处理，从而提高效率。ZNS+的关键设计主要有：

（1）设备内区块压缩：ZNS+支持在 SSD 内部执行数据复制，以此加速段压缩过程。

（2）异步处理：ZNS+的区域压缩命令是异步执行的，即主机不需要等待命令完成便继续执行后续的 IO 请求，这有助于降低阻塞充分利用资源，从而提高整体性能。

（3）稀疏型顺序覆写：ZNS+支持稀疏的顺序覆写，只要地址是递增的（但可以不连续）则可进行拉链式的填补合并，从而就可以在不重置的情况下对目标区域进行覆写，大大提高了效率。

具体地，ZNS+引入了新的命令 zone_compaction 和 TL_open，其中 zone_compaction 用于请求设备级区块压缩，能充分利用硬件内的电路级优化加速压缩过程。而 TL_open 允许标记 Zone 为 TL_open 状态，表示 zone 接受了稀疏型顺序覆写。这些命令的引入都有效提高了硬件的整体性能。

此外，ZNS+的设计考虑了与日志文件系统的兼容性，并旨在减少主机端的计算资源和带宽占用。ZNS+的实现在 SSD 模拟器和真实的 SSD 上都进行了，并通过修改文件系统（如 F2FS）来充分利用 ZNS+的特性。性能提升的研究表明，ZNS+存储系统在文件系统性能上比传统 ZNS 有显著提升。

ZNS+的映射机制利用逻辑地址空间的划分和映射，通过 Zone ID、StripeID、ChipID 和块内偏移实现数据的精确定位。这种设计允许 ZNS+利用 SSD 的内部并行性，同时将存储维护的很多责任从 SSD 本身转移到了主机系统。

总的来说，ZNS+通过其先进的接口设计和特性，为提高 SSD 存储设备的性

能和效率提供了有效的解决方案，尤其是在需要高并行写入性能等对存储有特别需求的应用场景中。

1.2.2 eZNS：更灵活的 Zone 接口

eZNS（Elastic Zoned Namespace）是一种为提高 ZNS SSD（Zoned Namespace SSD）性能而设计的弹性分区命名空间接口。eZNS 旨在提供一个应用透明的、灵活高效的逻辑分区抽象层，以充分利用 SSD 的闪存并行性和开放分区资源，并减少多租户间性能干扰。

随着 SSD 取代机械硬盘成为主流存储设备，传统的块接口与 SSD 的闪存介质特性不匹配，导致性能波动和成本增加。ZNS 接口技术因此被提出，旨在解决上述这些问题。eZNS 的关键设计主要有：

（1）物理分区条带分配策略（Serial Zone Allocator）：避免物理分区分配时的重叠，提高并行性。

（2）动态的物理分区条带膨胀策略（Zone Ballooning）：根据负载需求动态调整条带宽度，优化写带宽和资源分配的公平性。

（3）阻塞感知的分区读写请求调度器（Zone I/O Scheduler）：通过拥塞控制和写缓存准入策略，减少多租户场景下的性能干扰。

该工作进行了系统、真实的评估。实验结果显示 eZNS 在吞吐量和尾部延迟方面取得了显著改善，eZNS 可以实现对 ZNS SSD 的透明使用，并弥合了应用程序需求与区域接口特性之间的差距。具体来说，与静态区域接口相比，eZNS 在吞吐量方面提高了 17.7%，在尾延迟方面提高了 80.3%。这表明 eZNS 在提高性能和降低延迟方面具有显著效果，对多种应用场景都有积极影响。

1.2.3 ZNSwap：ZNS 作为交换设备扩展内存

ZNSwap 是一种针对最新的 Zoned Namespace (ZNS) SSDs 进行优化的交换子系统。该工作探索了 ZNS 作为内存交换设备的应用可能，一方面利用了 ZNS SSDs 提供的粗粒度分区抽象来增强成本效率；另一方面，它利用 ZNS 在驱动器上对数据管理的显式控制，引入了一个集成于操作系统内存交换逻辑的高效主机端垃圾回收器（GC）来优化内存交换。此外，ZNSwap 实现了跨层优化，例如 GC

时，其能直接访问 Linux 内核中的交换分区的使用统计信息，以实现细粒度的交换存储管理，并在操作系统资源隔离机制中正确计算 GC 带宽使用情况，以提高多租户环境中的性能隔离。

具体地，ZNSwap 的核心设计思路是利用 ZNS SSD 的特性，将每个区域（zone）配置为一个交换分区（swap partition）。传统的交换设备通常是基于块设备的，而 ZNS SSD 则是基于区域的。因此，利用 ZNS SSD 作为交换设备需要重新设计交换系统以适应 ZNS SSD 的特性。于是，ZNSwap 引入主机端的垃圾回收器（Garbage Collector, GC）集成进 Linux Swap 的交换逻辑中，从而提高了在主机端对数据管理的控制，实现了一个高效的交换分区设备，适用于大规模的内存扩展。

通过使用标准的 Linux 交换基准测试和两个生产级键值存储系统的评估，ZNSwap 展示了与传统 SSD 上的 Linux 交换相比的显著性能优势，例如在不同内存访问模式下具有稳定的吞吐量，并在实际使用场景下，99th 百分位延迟降低了 10 倍，吞吐量提高了 5 倍。

1.3 论文的主要内容与结构

1.3.1 论文的主要内容

（1）ZNS SSD 所需的软硬件环境

（2）RocksDB 及其对 ZNS 的兼容工作的瓶颈分析

（3）ZAFS 的设计与实现

（4）ZNS 基本测试与 ZAFS 性能验证

1.3.2 论文结构

本文的主要内容如下：

在第一章中，首先介绍了选择研究 ZNS SSD 的背景和意义，分析了国内外在该领域的研究进展，概述了本论文的研究内容和采用的技术路径。

第二章介绍相关技术基础，讨论了几种新型 SSD 和 ZNS SSD 的联系，随后阐述了键值存储和基于树的 LSM-Tree 模型的设计理念，以及传统 Linux I/O 栈中存在的问题。

第三章详细介绍了 ZNS 所需的软硬件环境依赖；随后剖析了 RocksDB 的设计与实现，以及 RocksDB 对 ZNS 的适配进展，发现了现有设计存在的问题；最后提出了写前日志的优化方案，并描述了 ZAFS 的设计与实现。

第四章对 ZNS SSD 进行了基准测试，并通过对 ZAFS 的评估验证了第三章中提出的设计的有效性。

第五章总结了上述工作并展望了基于 ZNS 的存储系统的前景。

2 相关技术基础

在本章节中，我们首先介绍几种稍早于 ZNS SSD 的新型 SSD 技术；随后我们介绍 ZNS SSD 目前相对成熟的应用领域——键值存储与 LSM-Tree 模型，最后我们将自顶向下地介绍本文将涉及到的 Linux I/O 栈中的关键概念和技术，并讨论传统 I/O 栈存在的问题。

2.1 几种新兴 SSD 技术

2.1.1 Multi-Stream SSD

Multi-Stream SSD[9]技术是一种旨在提升 SSD 性能的创新方案。通过引入对写入通道（即数据流）的精细控制，允许操作系统根据数据的特性和预期寿命主动将数据分散存储在 SSD 的指定区域。这种方法使得每个数据流能够独立管理，减少了不同数据类型之间的相互干扰，从而显著降低了写放大效应，并提升了 SSD 的耐久性和整体性能。

Multi-Stream SSD 技术特别适用于高写入负载和数据生命周期特征明显的应用场景。通过将这些数据分配到不同的数据流，Multi-Stream SSD 可以减少不必要的数据迁移，提高整体性能。

尽管 Multi-Stream SSD 提供了诸多优势，但它需要操作系统和 SSD 固件的支持才能充分利用其特性，因此，使用该技术也为操作系统和应用开发者带来了额外的工作负担。

2.1.2 Open-Channel SSD (OC SSD)

开放通道固态硬盘（Open-Channel SSD[10]）是一种新型的存储设备，它在传统固态硬盘的基础上进行了较大的改动。与传统 SSD 相比，OC SSD 的最显著特点是取消了传统的闪存转换层（Flash Translation Layer，FTL），将数据管理的责任转移到了主机端。这意味着操作系统可以直接管理 SSD 的闪存资源，从而实现更灵活、更精细的数据管理和优化。

OC SSD 的主要优势在于其允许操作系统更精确地控制数据的放置和回收过程，以适应特定的工作负载。通过优化垃圾回收过程，OC SSD 可以有效减少写

入放大效应（Write Amplification），从而延长 SSD 的寿命。此外，OC SSD 还可以实现 I/O 隔离，减少不同用户数据之间的相互干扰。

然而，OC SSD 也带来了新的挑战。使用 OC SSD 需要操作系统需要承担更多的责任，包括数据管理、磨损均衡和错误恢复等。这增加了操作系统的开销，并且与多流 SSD 类似地，OC SSD 要求文件系统和 SSD 固件之间的紧密协作，增加了开发成本。

2.1.3 ZNS SSD 与前置技术的关系

Zoned Namespace SSD（ZNS SSD）代表了对现有固态存储技术的进一步优化与发展。在传统的 SSD 技术如 Open-Channel SSD 和 Multi-Stream SSD 中，已经实现了对数据管理深层次优化，这些技术主要关注于改进数据的写入效率和减少写入放大现象。例如，Open-Channel SSD 通过取消传统的 Flash Translation Layer（FTL）并将数据管理的责任转移到主机端，使得数据存储过程更加透明，允许操作系统执行更精细的数据放置和回收策略。而 Multi-Stream SSD 通过引入多数据流的概念，允许系统根据数据的特性进行分区管理，有效地降低了数据干扰和写入放大。

ZNS SSD 在这些技术的基础上进一步引入了基于区域的存储模型，并且直接取消了 FTL 层的设计，放弃了随机写的支持，进而消除了盘内写放大效应，提高了写入操作的可预测性和效率。此外，ZNS SSD 的区域化管理降低了垃圾回收的复杂性和频率，显著提升了 SSD 的整体性能和耐用性。

ZNS SSD 在 OC SSD 基础上，向着提高存储效率和增加设备使用寿命的目标迈进。上述技术都通过不同的机制改善了存储系统的性能，而 ZNS SSD 的引入可视为对这一系列技术革新的延续和补充。在面向未来的高效能数据中心和高性能计算环境时，上述技术的设计理念和实践尝试能够提供有价值的参考，且 ZNS SSD 已有硬件产品落地，有希望推广成为一种灵活且高效的存储解决方案。

2.2 键值存储与 LSM-Tree 模型

2.2.1 键值存储

键值存储（Key-Value Store）是存储领域的一种非常重要存储模型，在这种

存储模型中，数据被简单地组织为键值对（Key-Value Pairs），其中键（Key）是唯一的，用于标识和检索对应的值（Value），值是变长的复杂数据。这种直接映射的方式，使得键值存储在处理大量数据时能够提供快速的数据访问和高效的存储管理。

键值存储的简单性和普适性是其最显著的优势。由于键值对的直接映射关系，键值存储的接口通常非常简单，易于编程和使用。键值对的结构允许存储各种类型的数据，包括但不限于字符串、数字、对象等。利用键值存储结构构建基本的存储引擎，在其上可以开发各种类型的数据库，再供不同场景下的应用使用，这样避免了重复构建基础组件，只需维护一个高效的底层键值存储引擎即可。上述特性使得键值存储可以广泛应用于不同的应用场景，如缓存系统、会话管理、配置存储以及分布式系统等。

具体地，键值存储的概念可以基于内存或磁盘进行不同的实现。内存数据库如 Redis，利用高速内存和基于键值的底层设计提供了极其高效的数据访问速度，成为许多实时应用的首选存储解决方案。而磁盘数据库如 LevelDB，则通过优化磁盘操作，实现了高效的数据存储和访问。此外，如 Cassandra[22]和 Riak 等等数据库，都基于键值设计实现了高可扩展性，高可靠性高容错性的存储系统。

键值存储涉及的研究领域非常广泛，包括性能优化、数据一致性、容错机制、数据压缩和安全性等多个方面。在性能优化方面，研究者致力于通过各种技术手段，如内存优化、索引结构优化等，提升键值存储的读写性能；数据一致性是分布式系统中的关键，优秀的一致性模型能确保在分布式环境中数据的一致性和高可靠性。容错机制的研究则关注如何在节点故障的情况下，保证系统的稳定运行。数据压缩技术的研究旨在减少存储空间的需求，利用计算开销换取高存储效率。安全性研究则关注如何保护存储在键值存储中的数据，防止未授权访问和数据泄露。

2.2.2 LSM-Tree 模型

LSM-tree（Log-Structured Merge-Tree[21]）模型是一种设计巧妙的数据存储模型，广泛应用于键值型数据存储引擎。LSM-tree 模型旨在解决传统数据库在处理写入操作时所面临的性能瓶颈。其设计理念是不进行原地更新，而是通过日志式

的追加写入和层次化设计，将随机写转换为高效的顺序写，从而优化整体性能。但代价是读性能差，写放大效应显著，垃圾回收开销高。

具体而言，LSM-tree 不是树的一种，也不是某种具体的数据结构。而是一种存储架构设计模型，在 LSM-tree 模型中，数据在一个或多个树状结构中首先被写入最顶层。最顶层写满后，刷新到下一层（通常是持久化设备），该过程通常伴随着数据排序，以便后续的读取和合并操作。随着数据的累积，当某层中的数据达到一定大小时，会被合并并下压至下一层。合并过程减少了数据数量，删除了过时或重复的数据，释放了空间。合并策略的设计直接影响 LSM-tree 的性能，包括写入放大因子和读取性能。

尽管 LSM-tree 模型在提高写入性能方面表现出色，但是仍存在一些问题。例如，写入放大因子可能导致实际写入磁盘的数据量远大于原始数据量，在存储资源受限的环境中可能成为问题。此外，LSM-tree 将随机写转换为顺序写，是以牺牲读性能为代价的。随着数据积累，读取性能可能受到影响。为解决这些问题，研究者也提出了多种优化策略和技术，如布隆过滤器、层级合并策略、缓存机制和数据压缩技术。

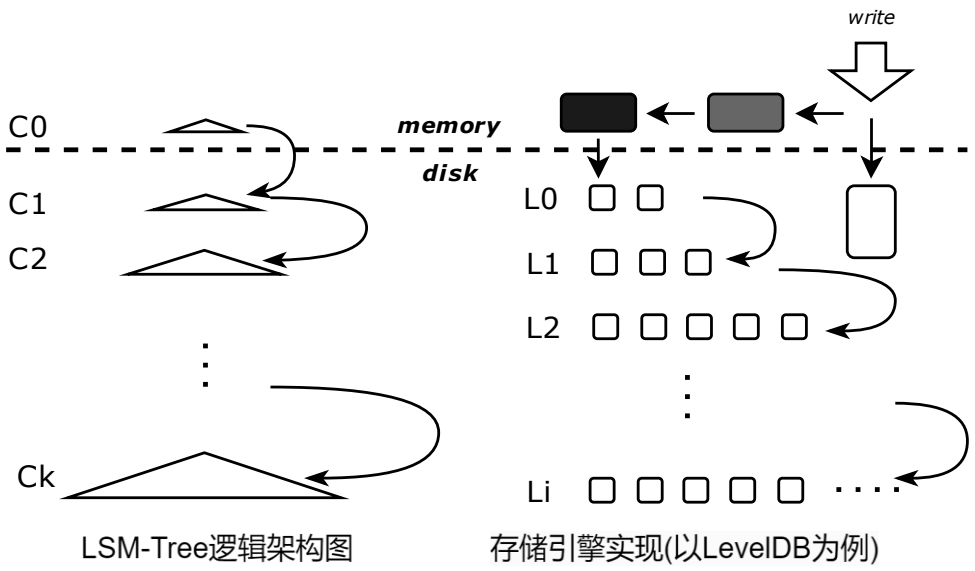

图 2-1 LSM-Tree 设计及实现的架构图

LSM-tree 模型以其在写入密集型应用中的优异性能成为现代数据库系统的关键组成部分，是最常见的存储引擎模型之一。并且随着各大公司对 LSM-tree 的

广泛使用，大大促进了该技术的进步，未来，对 LSM-tree 模型的研究和优化将继续深入，以满足不断增长的数据存储和管理需求。

在谷歌基于 LSM-tree 结构推出开源键值存储引擎 LevelDB 后，LSM-tree 名声大噪。对照 LSM-Tree 的设计与 LevelDB 的实现，如图 2-1 所示。

以 LSM-tree 模型构建的存储引擎还有 RocksDB 等等。RocksDB 是基于 LevelDB 优化的工程性实现，做了进一步优化，广泛用于各大互联网公司。

2.3 Linux I/O

2.3.1 I/O 基本概念

I/O（输入/输出）是指在主存和外部设备（磁盘驱动器、网络、终端）之间复制数据的过程。输入是从外部设备复制到主存，输出是从主存复制到外部设备。在 Linux 系统中所有的 I/O 设备都被映射称为文件，所有的输入输出都被当做相应文件的读和写来执行，所以内核提供了系统级的 I/O 函数接口，使得所有输入输出都以统一且一致的方式来执行。Linux I/O 栈（I/O Stack）是指在 Linux 操作系统中，从用户空间到内核空间，再到硬件设备，整个数据输入输出过程中所涉及的一系列组件和操作的集合。这个栈涵盖了从应用程序发起 I/O 请求开始，到数据实际在硬件上进行读写的完整流程。下面我们自上而下地介绍 Linux I/O 栈的各个层次中的关键概念或组件：

（1）用户空间（User Space）：用户空间是应用程序运行的地方。当应用程序需要进行 I/O 操作时（如读写文件、网络通信等），它会通过系统调用（system call）向操作系统内核发出请求。系统调用是用户空间与内核空间之间通信的桥梁。

（2）系统调用接口（System Call Interface）：系统调用接口是内核提供给用户空间的一组函数，用于执行各种 I/O 操作。这些函数封装了复杂的内核操作，对用户空间透明。常见的系统调用包括`read()`、`write()`、`open()`、`close()`等。

（3）内核空间（Kernel Space）：内核空间是操作系统内核运行的地方，负责管理计算机的资源和提供各种服务。当收到系统调用请求后，内核会进行相应的处理，包括权限检查、资源分配等。

（4）文件系统抽象层（VFS - Virtual File System）：VFS 是 Linux 内核的一

个组件，它提供了一个统一的文件系统抽象层。无论底层是何种类型的存储设备或文件系统，VFS 都能以统一的方式处理 I/O 请求，简化了内核和应用程序的设计。

（5）设备驱动（Device Driver）：设备驱动程序用于管理特定类型的硬件设备。它负责将内核的通用 I/O 请求转换为特定硬件能够理解的命令，负责与设备交互。设备驱动还能应对硬件的特性和限制，如缓冲管理、错误处理等。

（6）I/O 调度器（I/O Scheduler）：I/O 调度器是内核中的一个组件，用于优化磁盘 I/O 请求的执行顺序。它实现在多队列块层中，通过设置特定算法（如完全公平队列 CFQ、Deadline 等）来优化 I/O 请求的执行。

（7）硬件抽象层（HAL - Hardware Abstraction Layer）：HAL 是内核与硬件之间的中间层，它提供了一组通用的硬件访问接口。HAL 隐藏了硬件的具体细节，使得内核和设备驱动可以以统一的方式与硬件交互。

（8）中断处理（Interrupt Handling）：当 I/O 操作完成时，硬件设备会通过中断信号通知 CPU。内核中的中断处理程序会响应这些中断，完成 I/O 操作的收尾工作，如更新缓冲区状态、唤醒等待的进程等。

（9）硬件设备（Hardware Devices）：硬件设备是 I/O 栈的最底层，包括磁盘、网络接口卡等。硬件设备负责实际的数据存储和传输工作。

了解了上述各层次的基本概念，我们可以将 Linux I/O 总结为如下几个功能模块：

（1）系统调用：提供了用户空间与内核空间之间的接口。

（2）文件系统抽象：提供了统一的文件系统抽象，不同的文件系统可以通过 VFS 对接到 LINUX。

（3）设备驱动：管理硬件设备，将 I/O 请求转换为硬件命令。

（4）I/O 调度器：优化磁盘 I/O 请求的执行顺序。

（5）中断处理：响应硬件设备的中断信号，完成 I/O 操作。

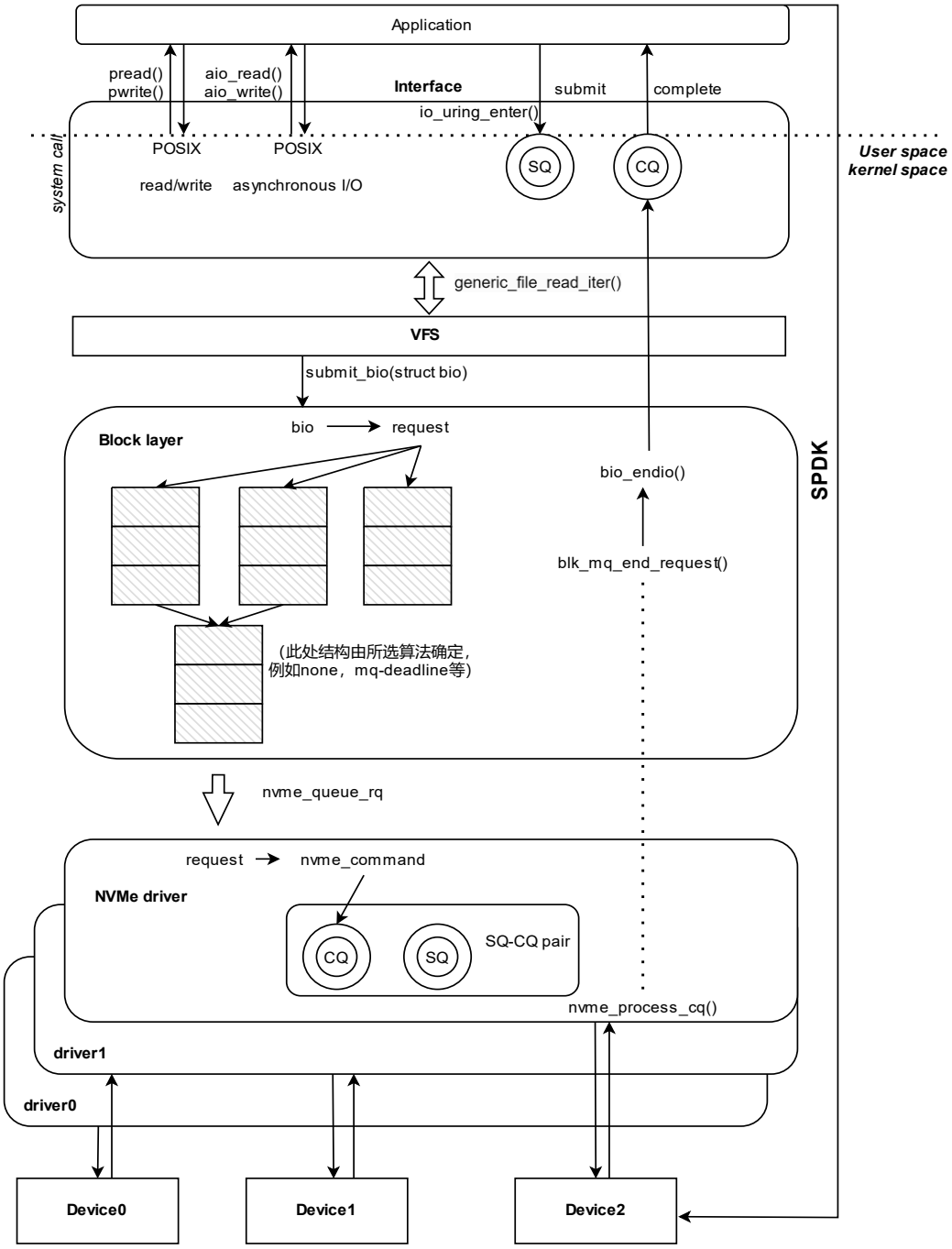

图 2-2 常用 Linux I/O 栈

2.3.2 现行 Linux I/O 栈

现有的常用 I/O 栈（驱动以 NVMe 设备为例）如图 2-2 所示。图中展示了多种 I/O 接口和方法，它们在现代 Linux 系统中提供不同的数据读写路径，其中比较重要的技术概念有：

（1）POSIX 同步 I/O ：POSIX 通用接口同步 I/O 是最基本的 I/O 形式，应

用程序通过如 pread()和 pwrite()等系统调用直接读写文件。这种方式下，应用程序会在 I/O 操作完成之前阻塞（等待）；对应的异步 I/O（AIO）通过 aio_read()和 aio_write()等系统调用实现。AIO 允许应用程序在 I/O 操作正在进行时继续执行其他任务，当操作完成时，应用程序会收到通知。这种方式减少了应用程序等待 I/O 操作完成的时间，提高了设备利用率。

（2）io_uring：io_uring 是 Linux 中相对较新的一种异步 I/O 接口，设计目的是为了提供高性能的异步 I/O 操作，尤其是在高负载环境下。它通过使用两个队列—提交队列（SQ）和完成队列（CQ）—来管理 I/O 请求和完成事件，极大地减少了系统调用的开销，并且提供了比传统 AIO 更灵活的操作方式。

（3）块层（Block Layer）：这不是一种应用层可以直接使用的 I/O 方式，而是操作系统内部用于处理磁盘块设备 I/O 请求的一种机制。它使用 bio 结构（块 I/O 结构）来管理磁盘 I/O 请求。块层还负责 I/O 调度，即决定如何排序和优化磁盘操作以提高效率和性能，使用的调度算法包括 bfq、kyber 和 mq-deadline 等。

（4）NVMe I/O：NVMe（Non-Volatile Memory express Zoned Namespaces）是一种专为固态存储设计的存储协议，通过 PCIe 接口与设备通信，比传统的 SATA SSD 有更低的延迟和更高的吞吐量。如图 2 所示，NVMe 驱动将 I/O 请求转化为 NVMe 命令，通过 PCIe 接口发送到 SSD，然后处理由 SSD 发送的中断来通知请求的完成。

每种 I/O 方式都有其特定的用途和优势，选择哪一种取决于应用程序的需求、系统性能目标和存储设备的特性。

2.3.3 传统 Linux I/O 路径中存在的问题

整个 I/O 路径涉及了很多层次，并且面对应用的需求越来越繁杂，最终使得现行的 I/O 栈非常冗杂。其中，传统的 I/O 路径为了保持其通用性，已经面临非常多的问题，严重影响了存储系统的性能。包括但不限于如下方面：

（1）性能瓶颈：传统 I/O 栈设计之初并未考虑到 SSD 的高吞吐量的特性，因此在处理高速数据传输时，使用传统的 I/O 栈可能成为整个系统的性能瓶颈。尤其体现在可能无法充分利用 SSD 的高 IOPS（每秒输入/输出操作数）能力，限制了其性能表现。

（2）高延迟与主机资源消耗：由于传统 I/O 栈在数据传输过程中涉及多次上下文切换和缓冲，导致整体延迟增加，这在对延迟敏感的应用中尤为突出。这些过程可能会消耗大量 CPU 和内存资源，尤其是在进行大量随机读写操作时。

（3）扩展性问题：随着存储设备容量的增加和分布式系统的普及，传统 I/O 栈在扩展性上存在限制，难以适应大规模存储系统的需求。

（4）缺乏硬件特性利用：现代 SSD 具有一些能够提升性能的硬件特性（如 NVMe 的队列特性），传统 I/O 栈未能充分利用这些特性，从而无法发挥 SSD 的全部性能。

（5）I/O 调度问题：在高负载情况下，传统 I/O 栈的调度算法可能导致 I/O 请求的不公平等待或者是调度本身开销过高，影响整体性能。

因此，高性能数据库和实时系统往往不使用通用 I/O，而可能更倾向于使用 io_uring 或更激进的 I/O 方式，以获得最低的延迟和最高的 I/O 吞吐量。

2.4 本章小结

在本章中，我们介绍了与 Zoned Namespace SSD（ZNS SSD）相关的一些关键技术基础。首先，我们探讨了几种新兴的 SSD 技术，包括 Multi-Stream SSD 和 Open-Channel SSD（OC SSD）。这些技术在提高 SSD 性能、降低写入放大效应和提升设备寿命等方面发挥了重要作用。

接着，我们介绍了键值存储与 LSM-Tree 模型，这两者是现代存储系统中常见的重要组成部分。键值存储模型简单而灵活，适用于各种应用场景，而 LSM-Tree 模型则在处理大规模写入操作时表现出色，成为许多数据库系统的存储引擎之一。

最后，我们深入探讨了传统 Linux I/O 栈存在的问题，包括性能瓶颈、高延迟、资源消耗等。这些问题严重影响了存储系统的性能和可扩展性，现代高性能存储系统往往都适配较新的解决方案，以选择高效的 I/O 路径。 ZNS SSD 作为新兴的存储设备，能选择的 I/O 路径有限，且需要慎重考虑和选择。

3 ZAFS 的设计与实现

在本章节中，首先基于对 ZNS 的调研阐述并分析使用 ZNS 所需的基本环境，硬件环境包括模拟器和真实设备；软件环境包括相关框架与工具库等。随后介绍 RocksDB 的实现及其对 ZNS 设备的适配，并分析现有工作存在的问题。最后给出了 A2-WAL 的设计与优化策略，并详细介绍 ZNS 专用的 ZAFS 文件系统的架构设计与实现。

3.1 ZNS 的基本环境

本小节阐述对 ZNS 的广泛调研，包含对 QEMU，FEMU 等模拟器以及最新的 NVMeVirt[23]模拟器的介绍与对比分析以及对现有 ZNS SSD 产品的参数介绍。

3.1.1 硬件环境

现已有几款模拟器在一定程度上支持对 ZNS SSD 的模拟。

QEMU（Quick EMUlator）是一个开源且性能优异的模拟器，能够模拟多种处理器架构和硬件设备。QEMU 通过动态二进制翻译技术，提供对客户操作系统的高效模拟，使其能够在宿主机上运行，包括模拟传统硬盘驱动器（HDD）以及固态硬盘（SSD）的行为。QEMU 作为一个通用平台，能被用以开发和测试各种底层软件，例如操作系统、文件系统和存储协议。

FEMU（Flexible EMUlator）是一种灵活的模拟器框架，允许用户自定义和扩展硬件模型以满足特定的模拟需求。FEMU 基于 QEMU 设计，但在模拟 SSD 设备方面提供了更细致的控制和更高的灵活性。相比 QEMU，FEMU 能够更精确地模拟 SSD 内部的操作，包括页面管理、块的擦写、FTL 算法和磨损均衡策略等等。此外，FEMU 支持多种 SSD 特性和配置，包括不同的闪存类型、接口标准和性能参数，使得研究人员能够在一个受控且可复现的环境中进行基于 SSD 的实验，这些特点使其成为研究新型存储技术和评估其性能的理想工具。

然而，QEMU 和 FEMU 作为通用模拟器，在模拟 SSD 设备时均存在一些局限性。尽管其设计初衷是提供对多种硬件设备的模拟，但在复现 SSD 的真实性能方面，如 IOPS 和延迟，QEMU 可能无法完全达到预期效果。此外，QEMU 在

模拟 SSD 的特定硬件特性，如 NVMe 接口的高级功能时，未能充分捕捉其性能和功能优势。写入放大效应和垃圾回收过程是 SSD 中重要的性能和寿命影响因素，QEMU 无法在这些方面上对 SSD 设备进行模拟。FEMU 是在 QEMU 基础上针对 SSD 设计的模拟器，一定程度上缓解了上述问题，提供了相对更高的准确性和可扩展性，但在模拟过程中也存在一些缺陷。一方面，由于 FEMU 仍是基于 QEMU 的工作，作为虚拟机在运行时会引入很多额外的性能开销，且会出现设备性能表现不稳定的情况。另一方面，FEMU 的安装和配置过程比较复杂，总体而言比较笨重。此外，FEMU 在模拟大容量 SSD 或高负载场景时对计算资源的需求较高，这对实验平台提出了更高的要求，限制了最大可模拟规模。因此，尽管 FEMU 在模拟 SSD 方面表现出色，但在模拟其他新型的存储设备或更复杂的存储系统时存在局限性。

NVMeVirt 是一款新型的轻量级 SSD 仿真器，与前两者的显著区别是，它通过内核模块来模拟各种类型的 NVMe 设备，例如传统 SSD、低延迟高带宽 NVM SSD、分区命名空间 SSD（ZNS SSD）以及键值 SSD。NVMeVirt 的设计目标是促进软件定义的 NVMe 设备，允许用户使用自定义功能来定义任何 NVMe 设备类型，并在软件中桥接主机 I/O 堆栈和虚拟 NVMe 设备之间的差距。与传统的 QEMU 和 FEMU 模拟器相比，NVMeVirt 提供了更高的灵活性和可定制性，允许用户定义具有特定特性的 NVMe 设备类型。作为一个内核模块，NVMeVirt 与系统的集成度更高，能够提供更接近真实硬件的性能模拟。此外，NVMeVirt 支持模拟包括传统 SSD、ZNS SSD 和键值 SSD 在内的多种高级存储配置，这些是现有模拟器不支持的功能。NVMeVirt 的设计使其成为存储系统研究的有力工具，可以用于研究数据库引擎的性能特性或扩展 NVMe 规范以提高键值 SSD 性能等研究工作。因此，NVMeVirt 模拟器特别适合需要定制化和高性能模拟的研究和开发场景。

鉴于传统模拟器存在的不足，本课题的虚拟环境基于 NVMeVirt 搭建。

在真实设备的制造上，目前，研制 ZNS SSD 设备的公司主要有西部数据，三星等。西数已推出 ZNS SSD 产品，ZN540，其部分公开的参数如表 3-1 所示。

表 3-1 西数 ZN540 部分参数

	Type 0	Type 1	Type 2	Type 3
Capacity	1024GB	2048GB	4096GB	8192GB
Form Factor	U.2			
Interface	PCIe 3.0 1x4 or 2x2			
Read (max MB/s, Seg 128KiB)	3200	3200	3200	3200
Write (max MB/s, seg 128KiB)	1200	1000	2000	2000
Read lOPs (max, Rnd 4KiB)	406K	390K	442K	486K
Read Latency (us, avg.)	73	80	80	90

ZN540 采用 QLC 颗粒，PCIE3.0 接口。据西数测试统计表明，相比于同介质的固态硬盘，ZN540 的寿命提升了 4 倍。

本实验基于浪潮 ZNS SSD 完成，其配置高于 ZN540：采用 TLCs 颗粒，PCIE4.0 接口。实验中使用的 ZNS SSD 及其具体参数参见第四章实验部分。

3.1.2 软件环境

本小节介绍几个 ZNS 所需的重要依赖库和工具。

libzbd[12]是由西部数据开发并维护的一个开源项目，旨在为 Linux 内核提供对区域型块设备(Zoned Block Devices，ZBD) 的支持。ZBD 是一种新兴的存储设备规范，它与传统的随机访问型块设备不同，它采用区域（zone）的概念来组织数据，每个分区支持顺序写入和批量擦除操作，这与传统块设备的特性，即每个块都可以独立随机读写，有所不同。这种设计特别适合固态存储技术，而 ZNS SSD 就是 ZBD 设备中最新最典型的一种。

具体地，libzbd 库作为内核模块实现，libzbd 的编译产物是一个 .ko 文件，它可以被加载到 Linux 内核中，从而扩展内核的功能集。在操作系统的架构中，libzbd 属于内核空间，它需要执行与硬件直接交互的任务，如 I/O 操作、设备状态管理等。内核模块提供了一种机制，允许操作系统内核在运行时动态加载和卸载功能模块，这为系统带来了更高的灵活性和可扩展性。我们可以通过引用 libzbd.a 使用该库中的函数以操作 ZNS SSD 设备。以下是对 libzbd 库中部分主要

函数及其功能的简要介绍：

（1）zbd_device_is_zoned(): 用于测试一个块设备是否为分区块设备。

这个函数可以帮助用户确定一个设备是否支持分区管理。

（2）zbd_open() 和 zbd_close(): 分别用于打开和关闭一个分区块设备。在使用该库进行操作之前，需要先打开设备，并在操作完成后关闭设备。

（3）zbd_get_info(): 获取一个已打开分区块设备的信息，包括设备的基本属性和分区的状态信息。

（4）zbd_report_zones() 和 zbd_list_zones(): 用于获取分区块设备的分区信息。可以获取所有分区的详细信息，包括分区类型、状态、起始位置、大小等。

（5）zbd_report_nr_zones(): 获取分区块设备的分区数量。这对于确定设备的容量和空间分布非常有用。

（6）zbd_zone_operation(): 执行分区管理操作，如重置写指针、显式打开或关闭分区等。

（7）zbd_reset_zones(), zbd_open_zones(), zbd_close_zones(), zbd_finish_zones(): 分别用于重置、显式打开、关闭和完成分区操作，从而实现对分区的灵活管理。

此外，libzbd 中还定义了一系列用于操作分区描述符信息的宏定义和工具函数，例如获取分区类型、状态、起始位置、大小、容量等信息，以及设置日志级别、获取设备模型、获取分区类型和状态的字符串描述等。

然而，在使用 libzbd 库时，需要格外注意线程安全性，因为该库不维护任何已经被打开的分区块设备的内部状态，需要由应用程序确保对分区的操作正确进行互斥。不过通过多线程应用程序中的同步机制来实现对分区的正确操作可以解决这一问题。

FIO（Flexible I/O Tester）是一款功能强大的开源测试工具，广泛应用于评估存储和网络设备的性能。它通过提供大量可配置的测试选项，允许用户模拟各种工作负载，测试存储设备的读写性能、IOPS、延迟等关键指标。

FIO 的设计注重灵活性和可扩展性，支持多线程和多进程运行，从而有效利用多核处理器的能力。并且，它能够针对文件和块设备执行测试，覆盖从单个存

储设备到复杂存储系统的广泛场景。该工具还具备实时监控功能，可以在测试执行期间提供性能反馈，并生成详尽的日志记录，便于后续的性能分析和结果验证。FIO 支持多种操作系统，包括 Linux、Windows 和 macOS，使其成为跨平台性能测试的通用解决方案。使用 FIO 时，用户通过编写配置文件来定义测试参数，然后通过 FIO 命令行工具执行测试。FIO 提供包含丰富统计信息的结果文件，为性能评估和系统优化提供数据支持。

从版本 3.30 开始，FIO 扩展了其功能，以支持 Zoned Namespace (ZNS) SSD 的测试。在使用 FIO 对 ZNS SSD 进行测试时，用户可以利用多种参数来定制测试案例，以便更准确地模拟和评估设备的性能。以下是一些关键的参数：

（1）ioengine：指定用于数据传输的后台引擎。对于 ZNS SSD，可以选择 psync ioengine，它支持与 zone 的顺序写入约束兼容的数据同步。

（2）offset_increment：这个参数用于定义每个作业（job）的起始偏移量增量。在 ZNS SSD 的上下文中，它通常与 zone 的大小相对应，确保每个作业都在不同的 zone 上操作。

（3）size：定义测试中使用的总数据大小。这个参数应与 ZNS SSD 的 zone 大小和测试需求相匹配，以避免跨 zone 的写入。

（4）numjobs：指定并行运行的作业数量。这个参数可以用于模拟多线程或多进程访问 ZNS SSD 的场景。

（5）job_max_open_zones：为每个作业设置最大的开放 zone 数。这个参数有助于控制每个作业可以同时访问的 zone 数量，以符合 ZNS SSD 的使用模式。

除了这些参数，FIO 还允许用户选择测试的 I/O 模式（如读写、随机/顺序）、块大小、以及是否启用直接 I/O 等。此外，FIO 的灵活性还体现在可以结合使用多个参数来模拟复杂的工作负载模式，如混合读写操作或特定的 I/O 深度（iodepth）。

在后续的实验中，将利用 FIO 选择合适的参数，进行多角度详细测试，并通过分析 FIO 生成的详细测试结果，以吞吐量、延迟、IOPS 等关键指标进一步评判 ZNS SSD 在不同工作负载下的性能表现。

3.2 现有方案的局限性与问题分析

本小节介绍 RocksDB 的读写流程以及在其基础上的对 ZNS 的适配工作——ZenFS。并阐述上述方案的局限性并分析其性能瓶颈。

3.2.1 基于固态硬盘的嵌入式存储引擎

过去十年间，基于固态硬盘的 SSD 在在线数据服务中被广泛应用。低延迟和高吞吐量的设备不仅支撑了软件充分利用其全部功能的能力，还改变了许多有状态服务的实现方式。SSD 在支持数百 MB/s 的带宽的同时为读/写操作提供了每秒数十万次的 IOPS，这可达传统磁盘的数千倍。然而，由于编程/擦除周期的限制，高写入带宽无法持续。这些因素为重新思考存储引擎的数据结构提供了机会。

无论是在延迟还是在吞吐量上，许多场景下，由于 SSD 的高性能，性能瓶颈从设备 I/O 转移到了网络。但是对于应用程序来说，其更倾向于将数据存储在本地 SSD 上而不是使用远程数据存储服务。而这增加了对嵌入在应用程序中的键值存储引擎的需求。

RocksDB 是一个高性能的持久化键值存储引擎，由 Facebook 基于 Google 的 LevelDB 代码库于 2012 年开发。该引擎专为固态硬盘（SSD）优化，设计用于满足大规模分布式系统的存储需求，并作为库组件嵌入到高级应用程序中。

RocksDB 沿用了 LevelDB 的 LSM-Tree（Log-Structured Merge-Tree）结构，与 LevelDB 相比，RocksDB 引入了多项优化和改进，尤其是在高并发写入需求的场景下。较为关键的改进有二，其一是降低了软件层面的写入放大，即每次写入操作引起的额外磁盘写入量，另一个则是通过优化压缩过程，有效提高了 RocksDB 写入效率。

因此，RocksDB 在处理大规模数据时表现出色，特别适用于需要高写入吞吐量和低延迟的场景。其特点包括：

（1）高性能写入：优化写入路径以降低写入延迟，并处理高并发写入请求。

（2）低写入幅度：通过压缩算法和数据结构设计减少磁盘写入影响。

（3）灵活的配置选项：提供丰富的配置选项以适应不同需求。

（4）丰富的功能：支持事务、压缩、备份和迭代器等高级功能。

RocksDB 自发布以来，不但被产业界长期广泛使用，同时还保持着很高的更新优化频率。例如在 Facebook 中，RocksDB 被应用于超过 30 个不同的应用程序，存储着数 PB 的生产数据。并且，除了作为数据库存储引擎，它还被用于流处理、日志/排队服务、索引服务和 SSD 缓存等多种服务。RocksDB 的可定制性使其能够适应各种工作负载，无论是为读性能、写性能、空间使用率，还是它们之间的平衡点，用户都可以根据自己的需求进行针对性调优。综上所述，RocksDB 以其高性能的写入、低写入幅度、灵活的配置选项和丰富的功能成为多种应用场景的首选存储引擎之一。

RocksDB 实例负责管理单个服务器节点上的存储设备数据，不涉及主机间操作，如复制和负载均衡，也不执行高级操作，如检查点等。这种统一的存储引擎相较于针对每个应用单独搭建自己的存储子系统带来了显著的优势：它避免了重复处理数据损坏、异常恢复、文件系统错误处理等问题；不同系统间可以共用一套基础设施，如监控系统、运维工具、调试工具等；底层的统一性也使得调试经验在不同应用间可以复用。

另一方面，RocksDB 是高度可定制的，因此它作为一个核心 KV 存储引擎能适应各种工作负载，用户可以根据自己的需要对它进行针对性调优。正是因为如此灵活的可配置性，RocksDB 不但可以作为很多不同数据库的存储引擎使用，如 MySQL，CockroachDB[24]，MongoDB，TiDB[25] 等，还可以满足流式计算（Stream processing），日志服务（Logging/queuing services），索引服务（Index services），缓存服务（Caching on SSD）等多种特性完全不同的业务需求。

3.2.2 RocksDB 的 I/O 流程

本小节分析读写流程并解析 RocksDB 的架构。

写入流程：数据写入过程如图 3-1 所示，首先将数据添加到内存中的 MemTable，同时为了数据恢复，所有更新也会记录到磁盘上的预写式日志（WAL）。MemTable 通常由跳表（SkipList）实现，该数据结构能在在平均 $O(log\,n)$ 的时间复杂度上维护数据的有序性，从而支持高效的插入和搜索操作。当 MemTable 达到预设大小限制时，它会被标记为不可变（immutable），并进行一个持久化到磁盘的过程。在这一过程中，MemTable 中的内容被持久化（flush）到磁盘上的

“排序字符串表”(Sorted String Table ，SSTable)数据文件中，并丢弃相关的 WAL。每个 SSTable 由多个均匀大小的数据块组成，每个数据块都配有索引块，以便进行高效的二分搜索。

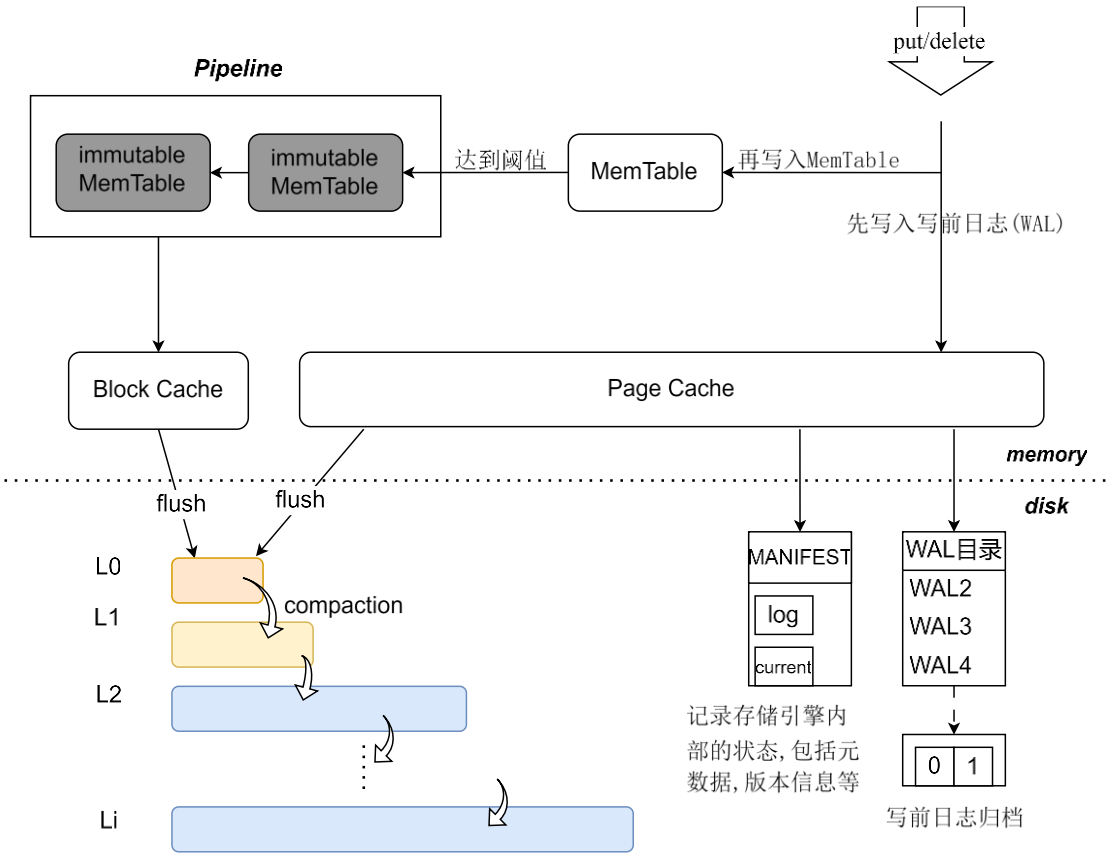

图 3-1 RocksDB 架构与写入流程

RocksDB 采用多层级的 LSM 树结构，如图 3-1 所示。随着时间的推移，数据在 LSM 树结构中积累。RocksDB 通过一个称为压实（Compaction）的过程把 SST 表合并并重新排序，从而创建出更高级别的 SSTable。压实操作会移除冗余数据，如已删除或被覆盖的键值对。压缩过程涉及垃圾回收和空间释放，RockDB 在这方面做了很多改进，在 3.2.4 节中会详细讨论。

读取流程：由于 LSM-Tree 的特性，RocksDB 不会覆写旧的数据，需要自顶向下扫描以确定某个键最新更新的值。因此读取操作需要从 MemTable 开始，逐步检查每个级别的 SSTable 直至找到第一条记录（即某个键对应的最新写入的值）。在按字典序排序后的单元中，RocksDB 能利用二分搜索迅速查找指定的键。为了进一步提高搜索效率，RocksDB 使用布隆过滤器来排除那些不必要搜索的

SSTable 文件。然而，如果需要执行范围扫描，仍必须搜索所有层级的 SSTable，进而导致很高的开销。

3.2.3 压缩（Compaction）过程

随着写入操作的进行，数据会逐渐积攒，并且由于 LSM 树中存在多个层级的 SSTable 文件（如 Level-0、Level-1 等），不同层级的 SSTable 文件可能存在重叠的键范围，产生大量过时且重复的数据。为了释放存储空间和提高查询效率，需要对这些 SSTable 文件进行合并和清理，也就是执行 compaction 这一过程。

在 RocksDB 中，compaction 是指将多个 SSTable 文件合并为更少且更大的文件，同时删除重复和过时的数据，并将数据重新组织以提高读取性能。具体来说，compaction 过程包括以下几个步骤：

（1）选择合并的 SSTable 文件：根据预设的合并策略和条件，选择需要合并的 SSTable 文件。通常是选择相邻层级的 SSTable 文件进行合并。

（2）合并数据：将选定的 SSTable 文件中的数据合并为一个新的 SSTable 文件。在合并过程中，会删除重复的键，并根据需要执行数据压缩。

（3）重新组织数据：重新组织合并后的 SSTable 文件，以便提高读取性能。这一阶段通常会对数据进行排序和分块，以便进行更高效的查询。

（4）清理过时数据：删除已合并的 SSTable 文件以及其中包含的过时数据，释放磁盘空间。

通过以上步骤，compaction 过程可以优化存储布局，减少数据重复和过时数据，提高查询效率，并释放磁盘空间。这对于处理大规模数据和提高系统性能至关重要。因此，RocksDB 对 compaction 过程做了很多优化。RocksDB 在 compaction 方面的改进相比 LevelDB 主要体现在以下几个方面：

（5）并行化多线程压缩任务：RocksDB 引入了并行化压缩任务的技术，使得多个压缩任务可以同时进行，充分利用多核 CPU 资源，提高了压缩效率。相比之下，LevelDB 的压缩任务是串行执行的，一次只能处理一个任务，导致压缩过程耗时较长。

（6）批处理压缩数据：RocksDB 采用了批处理压缩数据的策略，通过批量读写文件，这种方式相较于 LevelDB 在压缩过程中采用的逐个处理每个数据块

的方式，大大减少了磁盘访问次数。实现了高效的 I/O 操作。

（7）层级加权的压缩目标：RocksDB 采用了级别化的压缩目标，即不同层级内的 SSTables 具有不同的目标大小。这样可以根据数据的生命周期和访问频率，找到最优的计算-存储开销平衡点。而 LevelDB 在这方面的优化不如 RocksDB 灵活，缺乏动态调整压缩策略的能力。

RocksDB 与 LevelDB 二者在架构设计上大同小异，但 LevelDB 是机械硬盘时代的产物，而 RocksDB 则针对固态硬盘做了优化。除了压缩以外，RocksDB 针对 LevelDB 中的缺陷进行了多项改进和扩展，对其进行的多项改进和扩展具体包括：

列簇（Column Family）：RocksDB 引入了列簇的概念，允许开发者将相关的键值对分组存储在不同的列簇中。这使得数据的管理和访问更加灵活，同时可以对每个列簇独立地进行压缩和读取操作。

不可变内存表缓冲机制（Immutable Memtable）：RocksDB 维护多个不可变内存表，当一个内存表满了之后，它会被存入缓冲区，在缓冲区满后写入磁盘，而新的写入操作则在新的内存表中进行。这种机制避免了 LevelDB 中可能出现的写停顿问题。

TTL（Time-to-Live）机制：RocksDB 提供了数据过期机制，允许开发者为键值对设置生存时间，过期的数据将被自动清理。

分离的线程池：RocksDB 将数据的刷新（flush）操作和压缩（compaction）操作分配到不同的线程池中，并且刷新操作具有更高的优先级。这种设计可以减少写入延迟，提高写入性能。

Write-Ahead Log（WAL）管理：RocksDB 改进了 WAL 的管理机制，使得 RocksDB 能在崩溃恢复时更有效地使用 WAL。

多样化的压缩策略：RocksDB 支持多种压缩算法（传统意义上的数据压缩，并不是上文中的 Compaction），包括 Snappy、LZ4、Zlib 和 ZSTD 等，允许开发者根据数据特性和性能需求选择最合适的压缩策略，付出计算开销以节约空间。

自定义的压缩库和表格式：RocksDB 允许开发者使用自定义的压缩库和表格式，提供了更高的灵活性。

支持二进制和压缩的索引：RocksDB 支持二进制和压缩的索引，这可以减少索引的存储空间需求，提高查询效率。

事务和检查点：RocksDB 提供了对事务的支持，允许进行原子写入操作，并且可以创建数据的检查点。

通用的文件系统接口：RocksDB 抽象了底层存储，允许在不同的文件系统上运行，包括各类主流文件系统。使其成为适用性非常广泛的基础存储组件

跨平台支持：RocksDB 可以在多种操作系统上运行，包括 Linux、macOS 和 Windows。

3.2.4 RocksDB 对 ZNS 的支持

RocksDB 提供了一个灵活的插件机制，允许开发者根据自己的需求扩展数据库的功能。并且，RocksDB 通过其文件系统包装器 API 提供对独立存储后端的支持，该 API 是 RocksDB 访问其磁盘数据的统一抽象。从本质上讲，包装器 API 通过唯一标识符（例如文件名）识别数据单元，例如确定相关单元是 SST 文件还是预写日志 (WAL)。每个标识符除了随机访问，仅顺序写以及可按字节寻址的三种读写语义之外，还支持一组操作（例如，添加、删除、当前大小、利用率等）。这些操作构成了一个最小文件系统，是 RocksDB 的主要存储后端。通过使用该文件系统抽象，RocksDB 避免了管理文件范围、缓冲和空闲空间管理等工作。

因此，RocksDB 的存储插件提供的灵活性和文件系统包装器 API 带来的通用性使得适应不同的存储后端成为可能。

目前，ZenFS 实现了对 zoned 块设备的支持并集成到 RocksDB 中。ZenFS 是为 RocksDB 设计的，专门用于在分区存储设备（例如 ZNS SSD）上高效存储数据的文件系统。它充分利用了 RocksDB 的 LSM 树结构及其不可变的、仅顺序操作的压实过程，为分区存储设备提供了一种优化的数据管理方法。ZenFS 作为一个存储后端，实现了一个最小化的磁盘文件系统，并使用 RocksDB 的文件系统包装器 API 进行集成。ZenFS 适配 ZBD 设备的访问约束，与设备端区域信息元数据协作，为数据分配合适的区域。

具体地，它使用 RocksDB FileSystem 接口，将文件放置到原始 zoned 块设备

上的 zones 中。extent 是 ZenFS 中的最小管理单元，一个文件可以由一个或多个 extent 组成，所有 extents 构成的文件可以存储在设备的同一 zone（或不同 zones）中，但一个 extent 从不跨越多个 zones。 当区域中的所有 file extents 都无效时，可以重置该 zone，然后重新使用以存储新的 file extents。(即：文件可由一个或多个 extents 组成，且该文件可以存储在一个或多个 zones 中，但任何一个 extent 都不能跨 zone 存在)

数据放置：ZenFS 根据 RocksDB 库提供的“write lifetime hints”（WLTH） 将 file extents 放置到合适 zones 中。

垃圾回收：在 ZenFS 中，数据垃圾回收仅由 RocksDB 在启动 LSM-tree 表压缩过程时执行。ZenFS 和 ZNS 设备控制器都无需执行垃圾回收。

基于 ZenFS 的 RocksDB 能够利用 ZNS 降低系统写入放大，并且使得文件系统或设备上不需要后台垃圾回收，从而提高了整个系统的吞吐量、尾部延迟和 SSD 寿命等方面的效果。

具体地，ZenFS 的实现的主要功能如下：

（1）日志和数据分区的管理。ZenFS 定义了两种类型的分区：日志分区和数据分区。日志分区用于维护文件系统的状态，包括超级块数据结构和 WAL。数据分区则用于存储文件内容，它们被映射到一系列连续的区域，这些区域是按顺序写入的。

（2）超级块的作用与恢复。超级块是 ZenFS 从磁盘初始化和恢复状态的关键。它包含了文件系统实例的唯一标识符、魔术值和用户定义的选项。在系统启动或文件系统挂载时，超级块提供了恢复所需的基本信息。

（3）日志状态的维护与恢复。ZenFS 的日志负责维护超级块和 WAL 及数据文件到分区的映射。日志状态存储在专用的日志分区上，并使用特定的区域进行更新。恢复日志状态需要确定活动日志区域、读取日志头和应用日志更新。

（4）Zone 的分配。ZenFS 采用“write lifetime hints”（WLTH）算法选择最佳的区域来存储 RocksDB 数据文件。它按照文件的生命周期和区域内存储数据的最大生命周期进行分配，尽可能不增加 Zone 内数据的生命周期。

（5）Zone 写满阈值的自定义与性能优化。ZenFS 允许用户配置数据区域写

满的阈值，即指定区域剩余容量的百分比来控制文件大小和优化空间利用率的能力。此外，ZenFS 通过直接 I/O 和缓冲写入技术，一定程度保证了数据写入的效率。

ZenFS 使得 RocksDB 能够基于 ZNS SSD 设备运行且保持 RocksDB 的高性能特性和灵活性。ZenFS 的设计与 RocksDB 紧密集成，优化了数据的存储和访问过程，提高了系统的整体性能。

RocksDB 以 ZenFS 作为存储后端支持 ZNS 后，在吞吐量，延迟稳定性，寿命等诸多方面都取得了一定的进展，然而，RocksDB+ZenFS 的实现仍存在一些不足。

3.2.5 WAL 导致的写入瓶颈

写前日志（Write-Ahead Logging，WAL）是计算机科学中一种用于确保数据库系统原子性和持久性的技术。是指在应用修改之前将所有修改记录在日志中，为崩溃和事务恢复提供机制。现代文件系统通常使用 WAL 的变体，例如日志记录，用于元数据更新。

而对于 LSM-Tree 型存储引擎，使用 WAL 的具体情形如下：在写入易失性的 MemTable 时，如果发生电源故障，则无法保留数据。为解决这种由于断电等原因造成的数据丢失问题，存储引擎往往会在持久化存储设备中维护一个预写日志（WAL）。首先，将 put（写入）请求的记录存储在 WAL 中，然后将其写入到内存的 MemTable 中。在需要进行数据恢复的时候，按顺序读取所有 WAL 数据，依次执行对应操作来恢复内存中的 memtable。因为，在 LSM-Tree 中，只有对键值对的最新更改是有效的，所以 WAL 中的记录必须按顺序保存。

然而，这种设计可能导致频繁的小文件写入操作，成为性能瓶颈，因此，WAL 对于实现高写入吞吐量至关重要。一方面，因为 LSM 树通常部署在快速且高度并行的 NVMe 闪存 SSD 上，使用 NVMe 时，SSD 会发出内部管理操作，这些操作与 LSM 树竞争存储资源。这种竞争导致吞吐量不稳定。另一方面，在多线程环境中，还会发生锁争用，导致写入延迟。

解决上述问题的一种流行的方法是批量写入，RocksDB 中也实现了这一技术，具体如图 3-2 所示。该过程动态选择多个具有待处理 put 请求的工作线程中的

一个领导线程，收集所有剩余的记录，并让领导线程代表其他工作线程一次性写入它们。然而，这种技术并没有改善（甚至加剧了）尾延迟问题。在收集和写入这些记录的过程中，批处理中的所有记录共享写入时间，可能会增加单个写入的等待时间，尤其是当该条记录较大时。

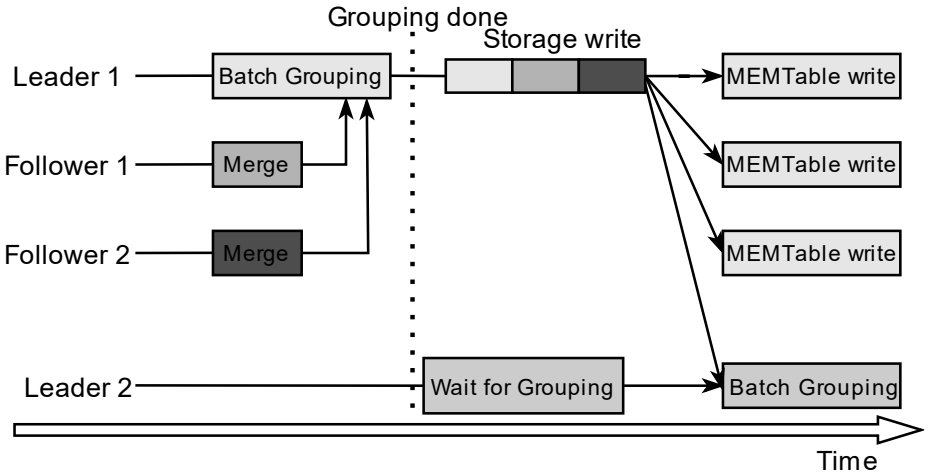

图 3-2　RocksDB 的批量写入时序图

ZNS 虽然能改善了 LSM 树吞吐量，使得延迟更稳定，但它对 LSM 树的 WAL 组件等其他组件造成了严重的写入效率挑战。主要原因是 ZNS 严格禁止随机写，禁止应用程序同时向同一区域发出写入 I/O。且 WAL 往往集中存储在单一区域内，这导致 WAL 的写入操作是串行化的，仅有一个写入操作同时处理这限制了 WAL 的吞吐量，进而影响了整个存储引擎的并发写入效率。另一方面，批量写入的解决方案本身带来的长尾延迟效应，削减了 ZNS 带来的延迟降低效果。因此，ZNS 没有展现出应有的性能优势。

3.2.6　Zone Append 命令

为了解决写入 I/O 的吞吐量限制，ZNS 引入了一种称为区追加的操作。Zone Append 命令允许主机系统将数据附加到指定区域（Zone）的末尾，而无需在写入之前执行擦除操作。这与传统的随机写入方式不同，后者通常需要先擦除闪存单元，然后才能写入新的数据。Append 命令最大的特点在于，不会在 I/O 提交请求中指定 LBA，而固态硬盘会在处理时确定 LBA 并在响应中返回地址。也就是说 Zone Append 命令通常是异步执行的，即应用程序在发送数据追加请求后

可以立即继续执行其他操作，而不需要等待数据写入完成。

因此，Zone Append 命令有很好的并发性，能有效增强了小文件顺序写入的吞吐量，使其成为实现 WAL 的可能的替代方案，能够解决高并发小文件（如上文的 WAL）的写入瓶颈问题。然而，使用 Append 命令仍需克服一些问题，由于 Append 需要在响应中才返回地址，也就是说，同时到达的写入命令的实际写入顺序是由设备决定的，可能与命令实际发出的顺序相违背，实际存储的是乱序结果。当系统崩溃，需要根据 WAL 恢复数据时，读取到的是无效的乱序结果。于是，我们通过为每个追加操作添加标识符以及一种基于重排序的恢复技术来解决 WAL 乱序写入问题。

3.3 ZAFS 架构概览

本课题基于 SPDK 实现了一个 ZNS 专用文件系统——ZAFS（Zone Append File System）。总体架构如图 3-3 所示。

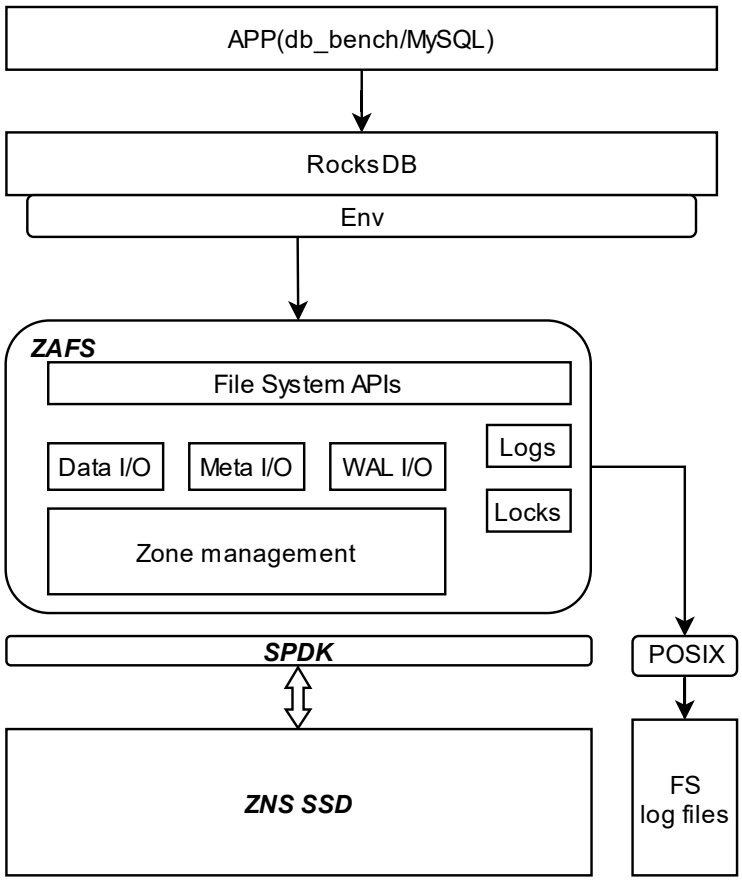

图 3-3 ZAFS 的架构

ZAFS 引入 SPDK 绕过 Linux kernel 以直接管理 ZNS SSD，有效降低了延迟；并且实现了一种区域友好的追加感知 WAL 解决方案——A2-WAL，解决了 ZNS 盘内只能串行写入 WAL 造成的写入瓶颈问题。

3.4 追加感知型 WAL

在 ZAFS 中，我们设计了一种适用于 ZNS SSD 的新型 WAL——A2-WAL（Append-Aware WAL）。并且，这种针对 WAL 的优化设计不仅限于 LSM-Tree 型的存储引擎，对于其他使用 Zone 语义存储 WAL 的数据库，A2-WAL 都能发挥作用。但是这些数据库对 ZNS SSD 的支持不够完善，因此，我们将讨论限制在基于 ZNS 的 LSM 树型存储引擎 RocksDB，并使用 ZenFS 作为参考模型，以解释我们为什么以及如何为 ZNS 更改现有的 WAL 设计。后续，我们仍基于 RocksDB 的场景下进行讨论，实现 A2-WAL 并集成在我们的 ZAFS 中。

首先，A2-WAL 设计的出发点是基于对 WAL 特性的三点观察：

（1）WALs 的写入请求是写入密集型，并主要发出小型写入；

（2）WALs 仅在数据库恢复期间读取；

（3）WALs 通常很小。

WAL 的写入操作非常频繁，且写入操作时，由于存储设备对小写入的限制，遇到了性能瓶颈；但是读取操作的发生频率很低，往往在数据库异常关闭，在下次启动时，才会发生数据恢复，进行 WAL 的读取。因此，我们认为以降低读取性能为代价来增加写入性能是可以接受的。ZAFS 的设计目标是提高 WAL 写入性能与并发性，充分利用区追加指令的峰值性能。消除 WAL 只能串行写入带来的性能瓶颈，从而提高存储引擎 Put 操作的效率。

在 WAL 中，有四个主要操作：写入，恢复，分配，删除。

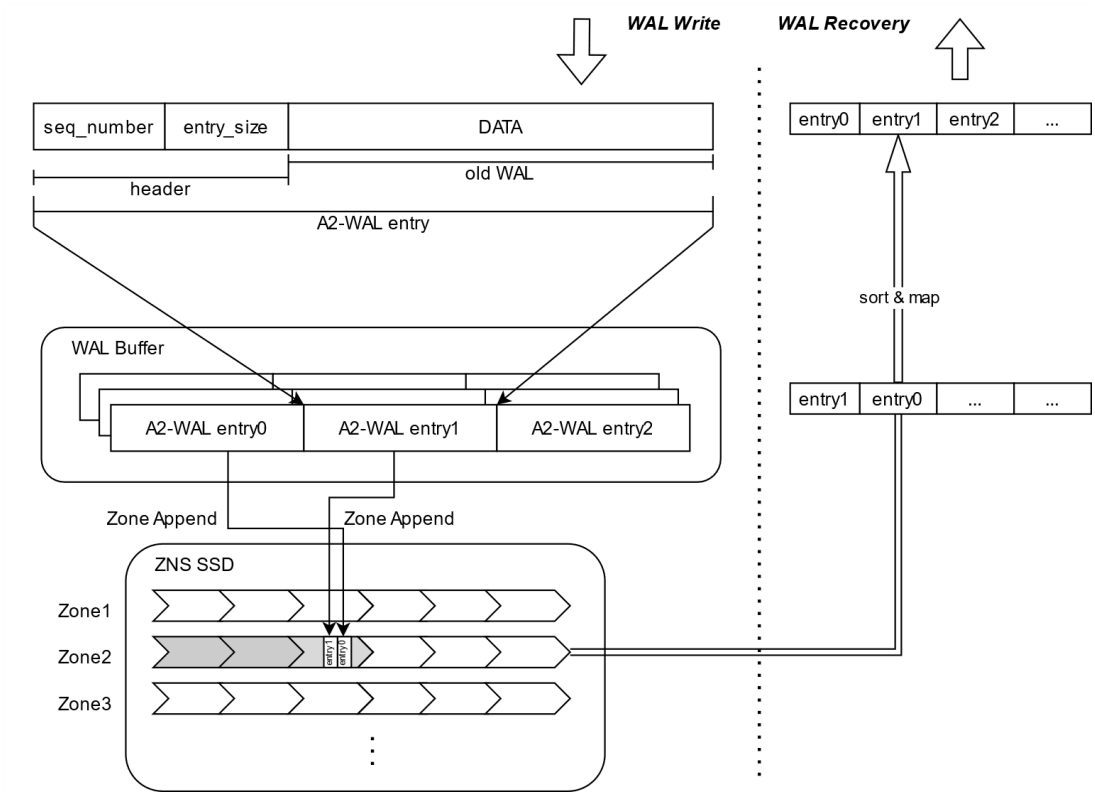

图 3-4　A2-WAL entry 的字段结构及其写入、恢复流程

3.4.1　WAL 写入

WAL Header：Zone-Append WAL 通过在每个 WAL 写入之前添加一个报头（128 位的 header）来帮助重排序。我们将 WAL 数据和头部组合成一个 WAL entry。这个报头(header)由 64 位序列号（seq_number）和 WAL entry 的大小(entry_size)组成。序列号以原子方式增加，并表示绝对数据顺序。条目的大小用于推断后续追加的 WAL 条目（如果有）的位置。如果使用区域追加命令返回的地址来确定数据存储的位置，这个返回地址是易失的，还需要另一个写操作将此地址存储到存储器中。

WAL Buffer：由于追加操作的最佳请求大小可能与页面大小不同，并且 键值 对可能明显小于 page size。因此，A2-WAL 的允许进行缓冲=。在写入 WAL 时，WAL 首先将数据复制到缓冲区。一旦缓冲区已满或者 WAL 已经同步（例如，fsync、close），我们使用区域追加命令将数据并行写入 ZNS SSD 为 WAL 分配的 Zone 内。

3.4.2 WAL 恢复

在 WAL 恢复过程中，LSM 树会顺序扫描所有 WAL，并将读取的数据应用到其内存表中。然而，A2-WAL 存储在 SSD 上是无序的；A2-WAL 需要恢复其原始顺序。A2-WAL 通过创建从逻辑地址（即偏移量）到物理地址的映射来实现这一点。

如图 3-4 所示，在读取时，首先使用 WAL 报头中的信息找到与逻辑地址（单线程顺序写入情况下的正确地址）对应的 WAL 条目的物理地址。由于序列号（seq_number）是唯一且单调递增的， 所以有更高序列号的 WAL 条目都有更高的逻辑地址。具体来说，具有序列号 x 的 WAL 条目的逻辑地址等于序列号为 x-1 的 WAL 条目的逻辑地址加上其条目大小（entry_size）。依次，构建逻辑-物理地址映射。然后按照序列号排序，得到一组顺序的物理地址。

然而，检索 WAL 头部并不容易。因为 WAL 条目是变长的，A2-WAL 无法预先确定 WAL 头部的位置(除了第一个请求)。为了避免每次读取都顺序扫描整个 WAL，逐个条目地查找 WAL 头部。一种替代解决方案是一次性读取所有 WAL，但这需要开辟与 WAL 等大小的内存。因此，A2-WAL 采用了一种更高效的方案：使用屏障。A2-WAL 按预定义的间隔插入屏障。在屏障处，A2-WAL 同步所有区域追加操作（即等待所有操作完成）。请求在连续的屏障之间进行排序，即屏障之后的追加操作的最低物理地址严格高于屏障之前的追加操作的最高物理地址。在读取过程中，A2-WAL 首先找到最近的屏障，然后读取两个屏障之间的所有数据。然后，A2-WAL 根据逻辑地址创建 WAL 条目的映射，并根据序列号对映射进行排序。通过这种设计，A2-WAL 每次只需要读取和维护一个页面间隔的映射，从而将 I/O 和内存占用限制在可配置的上限内。此外，该映射还可以作为缓存，因为读取是顺序的，并且后续读取很可能出现在同一个屏障内。根据设计，预期 A2-WAL 的恢复成本略高于传统 WAL，并且与屏障大小成比例。

3.4.3 WAL 分配

在 ZenFS 中，这涉及将一个区域（zone）分配给 WAL。这个区域并不专门用于 WAL，而是可以与其他 LSM-tree 组件共享。但是，在 ZAFS 中，A2-WAL 有更严格的限制，它需要为 WAL 专门分配区域。

原因有二：首先，为了防止其他 LSM-tree 组件向与区域追加（zone appends）相同的区域发出写入请求，因为这会导致区域追加需要等待写入完成，从而失去了优势。其次，A2-WAL 的恢复过程要求所有 WAL 数据都要连续存储，因此需要专门的区域来确保数据的连续性。

3.4.4 WAL 删除

关于 WAL 的删除，在 ZenFS 和 A2-WAL 中都将其视为释放存储资源操作，与其他数据删除并无不同。由于 A2-WAL 有专用的区域集合，因此可以随时安全地重置（将该区域标记为重置），而不会影响其他数据。删除可以是主动的（即立即执行）或者是延迟的（即在需要释放存储资源时执行）。

3.5 ZAFS 的实现

3.5.1 SPDK 技术

在第二章“相关技术基础”部分，我们已经讨论了 Linux I/O 栈的冗杂低效已经成为存储系统的瓶颈。在这种情况下，SPDK（Storage Performance Development Kit）技术的出现为解决高性能存储系统的设计与实现提供了一种全新的思路。SPDK 技术的核心理念是尽可能地减少软件层面对 I/O 操作的干预，通过将数据的传输和处理任务尽可能地移到用户空间，以最小化对内核的依赖性。这种基于用户空间的 I/O 处理方式可以显著降低系统中断的频率，减少了 CPU 的上下文切换开销，并且提供了更直接、更高效的 I/O 通道。

具体地，SPDK 是基于 DPDK（Data Plane Development Kit）的高性能数据包处理能力构建的一个专注于存储性能优化的软件栈。在 SPDK 技术中，采用了一系列优化策略，包括零拷贝数据传输、基于事件驱动的异步 I/O 模型等，以进一步提升存储系统的性能和吞吐量。此外，SPDK 技术提供了丰富的 API 和工具集并保持着较高的更新频率，方便开发人员快速构建高性能、低延迟的存储应用程序。

SPDK_NVMe：在 SPDK 中，专门针对 NVMe（Non-Volatile Memory Express）设备进行了适配和优化。NVMe 是一种专为固态存储设备设计的接口协议，相较于传统的 SATA 和 SAS 接口，具有更高的带宽和更低的延迟。SPDK 在 NVMe

方面的优化体现在以下几个关键方面：

首先，SPDK 提供了针对 NVMe 设备的用户态驱动程序，使得应用程序可以直接与 NVMe 设备进行交互，无需经过操作系统内核的介入。这种用户态驱动程序的设计可以显著降低存储操作的延迟，并提高系统的响应速度。

其次，SPDK 通过一组 API 实现了对 NVMe 命名空间的管理。NVMe 协议允许将物理存储空间划分为多个逻辑分区，即命名空间（Namespace）。SPDK 提供了丰富的命名空间管理功能，包括命名空间的创建、删除和查询等操作，使得用户可以灵活地管理存储空间。

第三，SPDK 针对 NVMe 命令的处理实现了异步事件驱动的机制。这意味着应用程序可以并行发送和处理多个 NVMe 命令，而不会阻塞主线程，从而提高了存储操作的吞吐量和并发性能。

最后，由于 NVMe 设备支持多命令队列，SPDK 也基于该特性实现了多队列并发处理机制。这种设计可以最大限度地发挥 NVMe 设备的并行性能，进一步提升了存储操作的并行处理能力。

SPDK_NVMe_ZNS：NVMe 协议规范已经定义了对 ZNS 的支持，包括命令集和数据传输方式等方面的规范。SPDK 作为一个存储性能优化的软件栈，在支持 NVMe 设备时，直接包含了 NVMe 协议规范中对 ZNS 的定义，将其集成到 SPDK 中，从而实现了对 NVMe ZNS SSD 的支持。这是本文工作的关键支撑之一，ZAFS 依托次工具库完成对 ZNS SSD 的高效交互，整个代码实现中出现调用该库接口 300 余次。

在 SPDK（Storage Performance Development Kit）中，针对 Zoned Namespace（ZNS）的典型函数主要用于处理与 zone 相关的操作，例如读取、写入、重置 zone 等。以下是一些典型的 SPDK 中与 ZNS 相关的函数及其详细说明：

（1）spdk_nvme_zns_report_zones()

功能：这个函数用于获取特定命名空间中的 zone 报告。它可以列出所有的 zones 以及它们的属性，包括每个 zone 的状态、大小和写入指针等。

参数：需要提供命名空间（ns）、队列对（qpair）、用于存储报告的缓冲区（buf）、缓冲区大小（bufsize）、起始 zone 的 LBA（start_lba）、要报告的 zone 数量

（num_zones）、报告类型（report_type），以及完成回调函数（cpl_cb_fn）。

（2）spdk_nvme_zns_reset_zone()

功能：此函数用于重置一个 zone，使其回到可写状态。这通常在 zone 完全写满之后需要主动调用。

参数：包括命名空间（ns）、队列对（qpair）、zone 的起始 LBA（start_lba）、是否重置所有 zones 的标志（reset_all_zones）、完成回调函数（cpl_cb_fn）和回调参数（arg）。

（3）spdk_nvme_zns_zone_append()

功能：这个函数用于向 zone 中追加数据。在 ZNS 设备上，数据通常是以 zone 为单位进行写入的，而追加操作是 zone 写入的基本形式。

参数：需要指定命名空间（ns）、队列对（qpair）、数据缓冲区（buf）、zone 的起始 LBA（zslba）、要写入的 LBA 数量（num_lbas）、完成回调函数（cpl_cb_fn）、回调参数（arg）和附加的标志（flags）。

（4）spdk_nvme_ns_cmd_read()

功能：虽然这个函数不是专门用于 ZNS，但它可以用于读取 ZNS 设备中的数据。它负责从指定的命名空间和 LBA 位置读取数据到提供的缓冲区。

参数：包括命名空间（ns）、队列对（qpair）、数据缓冲区（buf）、起始 LBA（lba）、要读取的 LBA 数量（lba_count）、完成回调函数（cpl_cb_fn）和回调参数（arg）。

（5）spdk_nvme_zns_close_zone()

功能：此函数用于关闭一个 zone，使其变为不可写状态。关闭 zone 是一个重要的步骤，它允许垃圾回收机制后续回收并重用 zone。

参数：包括命名空间（ns）、队列对（qpair）、zone 的 ID（zone_id）和完成回调（cpl_cb_fn）。

（6）spdk_nvme_zns_zone_appendv()

功能：这是一个向量变体的 zone append 函数，允许一次性发送多个数据块到 zone 中。这可以提高写入效率，因为它减少了单个 write 命令的数量。

参数：与 spdk_nvme_zns_zone_append()类似，但接受额外的参数来处理多个数据块。

（7）spdk_nvme_zmalloc

功能：分配一块对 DMA 传输安全的内存，并初始化为零。

参数：size：要分配的内存大小；align：内存对齐的字节数，通常为 64 字节；ctx：分配上下文，可用于跟踪或状态管理；socket_id：指定内存应该分配在哪个 CPU 插槽的内存上；flags：指定分配特性的标志，如 SPDK_MALLOC_DMA。

spdk_nvme_zmalloc 是 SPDK 中用于分配内存的重要函数，它确保了分配的内存适用于 DMA 操作，这意味着 NVMe 设备可以直接通过 DMA 读取或写入这块内存。该函数还会将分配的内存初始化为零，这对于初始化缓冲区或避免遗留数据非常重要。使用 spdk_nvme_zmalloc 分配的内存通常需要与 spdk_dma_free 配对使用来释放，确保内存被正确管理，避免内存泄漏。

3.5.2 独立的 WAL Zone

在最初 ZenFS 的设计中，Zone 按用途分为两类，即元数据 Zone 和数据 Zone，并没有对 RocksDB 的其他组件产生的数据作单独隔离，例如 WAL 并没有放在独立的 Zone 内。为了优化 WAL 的写入，并保证进行重排序时能够按照偏移地址相加寻址正常，WAL 将独占区域，在 ZenFS 的元数据和数据区域之间预留了一组专用的区域，用于存储 WAL。

3.5.3 与 RocksDB 的兼容性

为了支持 A2-WALs 在 RocksDB 中的使用，我们修改了一个函数。RocksDB 在删除 WAL 时，RocksDB 首先将 WAL 移动到归档目录，并在稍后的时间点才彻底地进行物理删除。但由于 WAL 能使用的区域数量有限，暂存这些失效的 WAL 需要分配新的 Zone，这可能会导致空间不足，因此我们进行修改强制 RocksDB 立即删除旧的 WAL。

3.5.4 代码结构简介

ZAFS 的实现保持了原有 ZenFS 的目录结构。主要分为三个文件。

io_zenfs.h：定义了用于 RocksDB 的 I/O 相关的类和接口，包含了如 ZoneExtent 和 ZoneFile 等类的定义，这些类表示文件系统中的区域和文件，并且负责编码和解码数据，以及处理数据的读写。该文件还包括一些全局变量和

extern 声明，用于跟踪写日志和与底层环境交互。 包含如下关键类：

（1）ZenFS 类: 代表文件系统本身，包含挂载、从元数据日志中恢复、创建文件系统、列出文件系统、获取区域快照等方法。

（2）ZoneFile 类: 表示文件系统中的文件。

（3）ZenMetaLog 类: 处理文件系统的元数据日志记录。

（4）Superblock 类: 表示文件系统的超级块。

（5）FactoryFunc: 一个工厂函数，根据 URI 创建文件系统的实例。

基于上述类实现了如下主要功能：

（1）挂载: Mount 函数通过从最新的有效元数据区域中恢复来挂载文件系统。

（2）恢复: RecoverFrom 函数从元数据日志中恢复文件系统。

（3）创建文件系统: MkFS 函数创建具有指定辅助文件系统路径和完成阈值的新文件系统。

（4）列出文件系统: ListZenFileSystems 函数列出现有的 ZenFS 文件系统。

（5）区域快照: GetZoneSnapshot 和 GetZoneFileSnapshot 等函数获取区域和区域文件的快照。

io_zenfs.cc：它包含了第一个文件中声明的类和所需接口的具体实现。主要实现了 ZoneExtent 和 ZoneFile 类的方法，如数据编码解码、JSON 序列化、以及与 Zoned Block Device (ZBD) 进行读写等操作。还实现了一些用于管理文件和文件区域的功能，如合并文件更新、文件重命名等。

zbd_zenfs.cc：此文件实现了 Zoned Block Device (ZBD) 的相关操作，它提供了管理 ZNS 设备区域的方法。定义了 ZonedBlockDevice 类，这个类负责打开和关闭设备、分配和释放区域、以及管理区域的状态等。文件中包括了一些底层的 NVMe ZNS 操作，例如区域重置、追加写、同步等。同时，包含底层调用，比如 SPDK 相关的代码来直接与 NVMe 设备进行交互，避免操作系统的存储栈。

3.6 本章小结

在本章中，我们探讨了 ZNS SSD 的特性及其对数据库管理系统，尤其是 RocksDB 的影响。我们首先介绍了 ZNS SSD 具体实施时所需的软硬件环境。随

后，我们明确了 RocksDB 的具体架构和读写流程。深入分析了针对 ZNS 设备实现的 RocksDB 的适配，分析了该实现仍存在的不足。

还讨论了 Zone Append 操作的实现及其对于数据库事务处理性能的提升，以及通过 Zone- Append WAL 来进一步优化日志写入的效率。最后，我们提出了 ZAFS，基于 SPDK 实现了 RocksDB 文件系统后端，并集成了 Zone- Append WAL 的方案，解决了 ZNS 中 WAL 串行写导致的写入瓶颈问题。整体来看，本章提供了对 ZNS SSD 在数据库环境中应用的深入分析，并指出了当前技术的限制并展示了未来发展的潜力。通过这些技术的集成与优化，能够显著提高大规模数据处理环境中的存储效率和系统稳定性。

4 性能测试与分析

4.1 实验平台配置

实验平台使用配有双路 CPU，大容量 DDR4 内存的高性能服务器，并装有两块浪潮 Inspur NS8600G2-Z ZNS SSD，其具体配置如表 4-1 所示。

表 4-1 实验平台配置

	配置
CPU	Intel(R) Xeon(R) Gold 6326 CPU @ 2.90GHz * 2
内存	503GiB
SSD	Inspur NS8600G2-Z
操作系统	Ubuntu 22.04.3 LTS（Linux 5.15.0）

4.2 模拟器与真实设备的对比评估

ZNS SSD 中，有一些重要参数在生产时确定，是不可更改的，这些参数与内部结构高度相关。例如 Zone Size，目前，在区域大小的设置上有很多争议，西数设计的区域大小在 GB 级，而三星设计的 Small-Zone ZNS 的区域大小只有 96MB。像这样的参数对比只能通过模拟器来测试。

实验中使用 FIO-3.28，该版本已支持 ZNS 的测试。

模拟平台使用 NVMeVirt 实现，模拟参数参考 WD_ZN540。

4.2.1 FIO 写入测试

写入负载：块大小为 64KB，16 个并发线程，每个线程最多只拥有 1 个 Zone。对比结果如表 4-2 所示。

表 4-2 NVMeVirt 模拟器评估

	写入速度	平均延迟
NVMeVirt	2626MB/s	75us
Inspur	5315MB/s	153us

模拟器的误差较大：根据上述实验结果可以观察到，模拟器与真实硬盘的差

距较大。通过阅读 NVMeVirt 原文，发现其支持模拟的最大 NVMe link 带宽达不到 PCIe4.0 水平，而我们使用的物理设备是 PCIe4.0 SSD，这是二者写入速度差距较大的直接原因在延迟方面，由于该模拟器以内核模块的形式实现，相较于其他模拟器没有虚拟机开销，但是，由于它的数据只写入到内存中，相较于真实设备，延迟过低，且只能通过设定一个固定值来弥补这个问题，这导致其对延迟的模拟非常局限。我们测试了在其他参数配置下模拟器的性能表现，均存在一倍以上的差距。因此，在硬件基准性能测试中二者差距较大，可见模拟器在性能上往往不能精确反映设备的真实情况。

4.2.2 RocksDB + ZenFS 应用测试

基于 RocksDB + ZenFS 分别对模拟和物理环境进行测试，记录吞吐量随时间的变化。

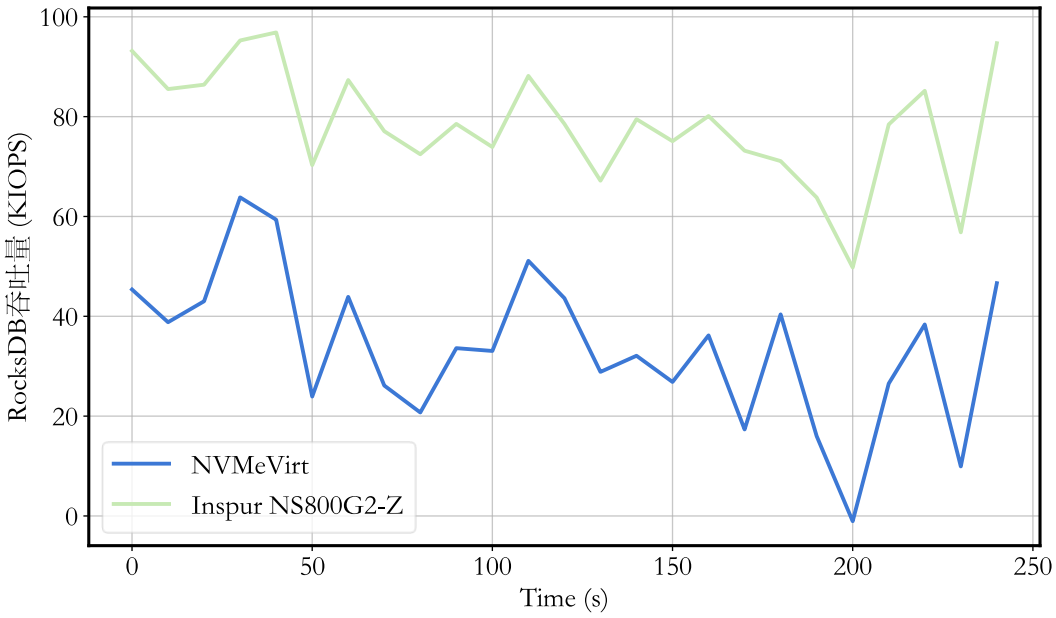

图 4-1 ZNS 模拟器与 ZNS 设备的吞吐量波动情况对比

从上图中可以观察到，尽管二者数值上差距较大，但是 NVMeVirt 的吞吐量随时间变化的趋势与真实设备基本一致，该模拟器能较好地观察设备的性能波动情况，有助于分析程序中的性能瓶颈。

4.3 基于 Inspur NS8600G2-Z 的 ZNS SSD 评估

4.3.1 设备参数

根据/sys/block/nvme0n1/queue/ 路径下的硬件信息，Inspur NS8600G2-Z 的详细参数，其中一些是固件参数，一些是可设置的参数，一些重要的参数见下表。

表 4-3 Inspur NS8600G2-Z 设备参数

参数	值	意义
hw_sector_size	4096	硬件扇区大小，以字节为单位。
logical_block_size	4096	逻辑块大小，以字节为单位。
max_active_zones	16	可以同时处于激活状态的最大区域数。
max_open_zones	16	可以同时打开的最大区域数。
max_sectors_kb	256	单次 I/O 操作可以处理的最大扇区数，总大小为 1MB
minimum_io_size	4096	最小的 I/O 操作大小，以字节为单位。
nomerges	0	不允许合并 I/O 请求以提高效率
physical_block_size	4096	物理块大小，以字节为单位
read_ahead_kb	128	预读操作的字节数
scheduler	[none]	当前使用的 I/O 调度策略。
virt_boundary_mask	4095	虚拟地址边界掩码，用于限制 I/O 请求的地址范围。（不可跨 ZONE 寻址）
wbt_lat_usec	2000	写回缓冲区（Write-Back Tuning）延迟时间，以微秒为单位。
write_cache	Write through	写入缓存策略为直写模式，即盘内无 Cache
zone_append_max_bytes	262144	每个区域追加模式下可以写入的最大字节数。
zoned	Host	区域管理方式为主机管理。
zone_write_granularity	4096	区域写入粒度，以字节为单位。

4.3.2 单 Zone 写入测试

在实际应用中，会出现某类型的数据仅写入单个指定 Zone 的情况（例如上文分析的 RocksDB 中的 WAL 文件），因此，测试不同情况下单个 Zone 的性能

很有意义。分别使用三种 I/O 引擎，在不同的块大小下进行单 Zone 写入测试，结果如图 4-2 所示。

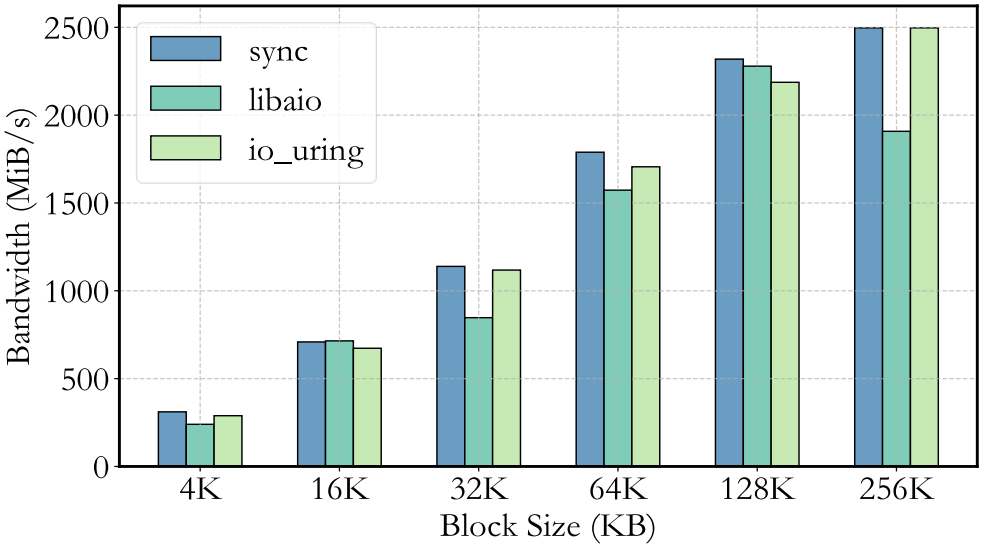

图 4-2 ZNS 设备单 Zone 写入速度测试

写入随着每次请求的块大小增大而增大，单个 Zone 的写入速度最大达到 2500MiB/s。这与全盘的写入速度差距较大，因为只使用一个 Zone 不能达到该盘最大带宽。

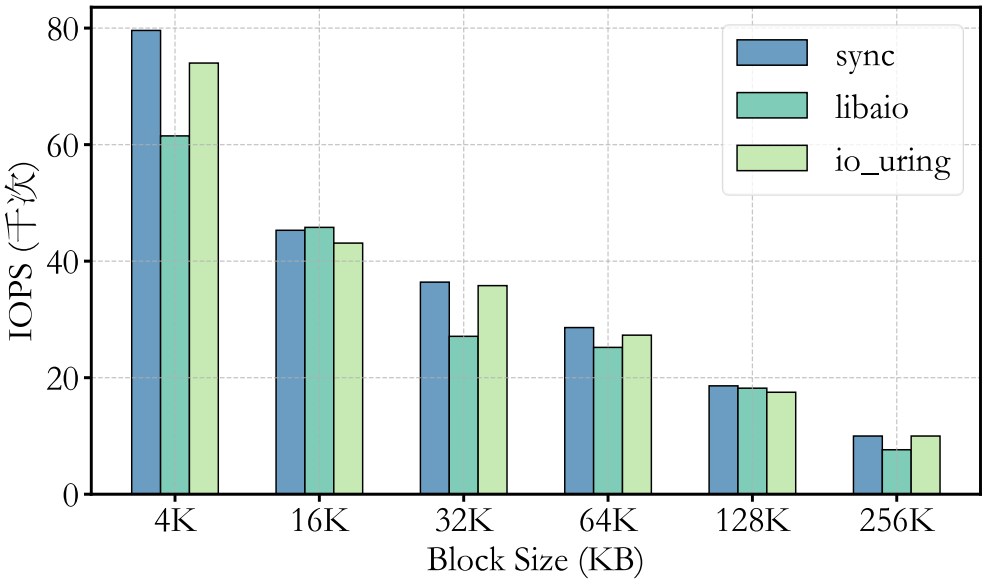

图 4-3 不同 I/O 引擎下单 Zone 写入 IOPS 测试

在不同的块大小下三种 I/O 路径的单 Zone 写入吞吐量结果如图 4-3 所示。观察到单个 Zone 的 IOPS 在 4K 时最大。并且吞吐量随着每次请求的块大小增大而下降。

4.3.3 全盘多线程写入测试

为了评估不同块大小和线程数对写入速度和 IOPS 的影响，我们设计并实施了一系列测试。测试配置包括使 io_uring 作为 I/O 引擎，并在五种不同的块大小（4k、16k、32k、64k、128k、256k）和五个不同的线程数（1、2、4、8、16）下进行测试。

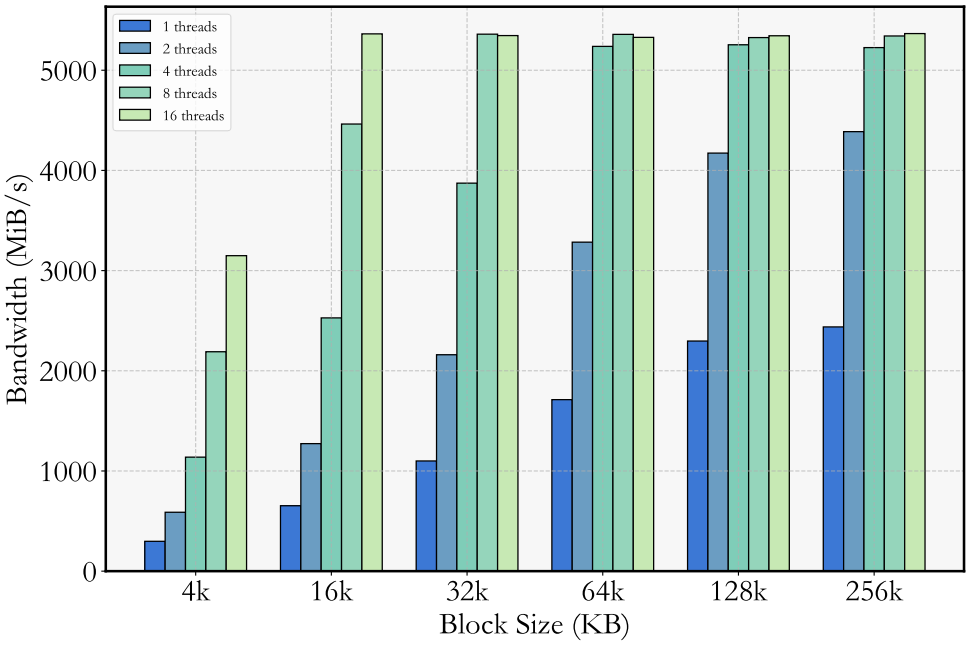

图 4-4 不同块大小的多线程全盘写入带宽测试

如图 4-4 所示，总体而言，写入带宽随块大小和线程数的增加而增加，符合预期中的结果。较小的块大小（如 4k）导致显著较低的带宽，这是由于处理更多 I/O 操作带来的额外开销。在块大小从 16k 逐渐增加到 256k 的过程中，带宽不断提升，但增长率逐渐减小，直至达到硬盘最大写入速度 5300MiB/s。线程数对带宽的影响在不同块大小下有所不同。在较小的块大小下（如 4k），增加线程数能够显著提高带宽，说明多线程能够更有效地利用 I/O 子系统的并行性。而在较大的块大小下（如 256k），线程数的增加对带宽的提升相对有限，因为此时磁盘带

宽已经接近饱和，多线程无法进一步显著提高性能。因此，实验结果表明，多线程和大块大小均有助于提升带宽，但在一定块大小和线程数之后，带宽的增长会受到硬件和系统瓶颈的限制。

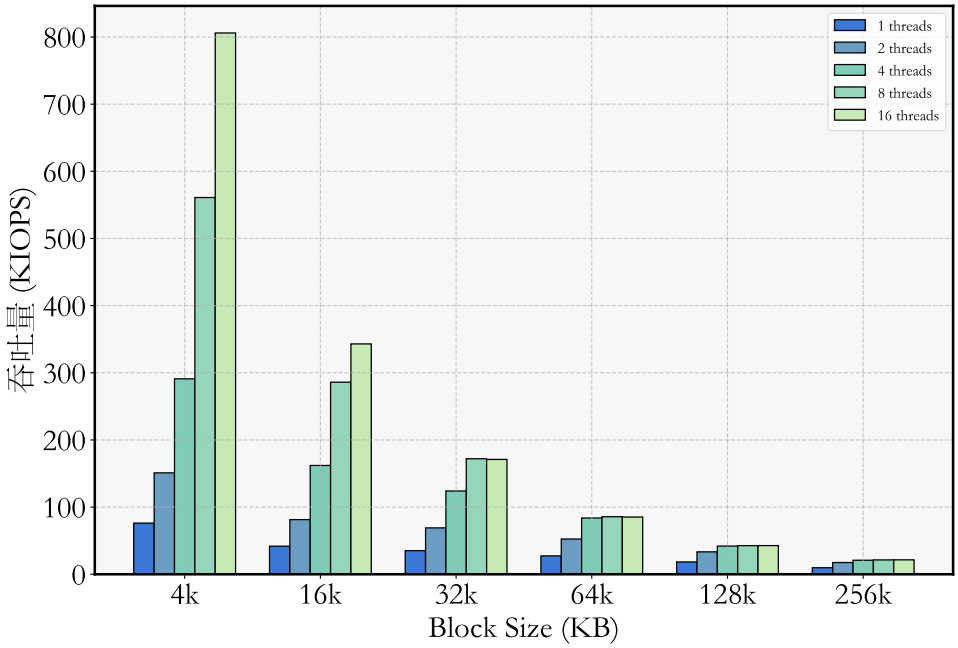

图 4-5 不同块大小的多线程全盘写入 IOPS 测试

图 4-5 中展示了不同块大小和线程数对吞吐量的影响。结果显示，4k 块大小下，IOPS 随线程数显著提高，从单线程的约 100 KIOPS 增加至 16 线程的约 800 KIOPS，表明在较小块大小下，高并发度显著提升 I/O 子系统的并行性和性能。16k 块大小下，IOPS 亦随线程数增加，从约 100 KIOPS 提高至 350 KIOPS，但提升幅度相较于 4k 块大小有所减小。随着块大小进一步增加至 32k、64k、128k 和 256k，IOPS 提升幅度逐渐减小，32k 块大小下从约 50 KIOPS 提高至 150 KIOPS，64k 块大小下从约 30 KIOPS 提高至 90 KIOPS，128k 块大小下从约 10 KIOPS 提高至 40 KIOPS，256k 块大小下从约 5 KIOPS 提高至 20 KIOPS。这表明在较大块大小下，多线程对性能提升的作用逐渐减弱，甚至达到瓶颈。因此，在需要高 IOPS 的应用场景中，选择较小的块大小和多线程配置可以显著提升性能，而在块大小较大的场景中，多线程对性能提升的效果有限。

4.4 ZAFS 评估

4.4.1 ZAFS 写入测试

利用 RocksDB 的 benchmark 工具，进行 4KiB 的 fillrandom 测试。测试结果如图 4-6 所示。

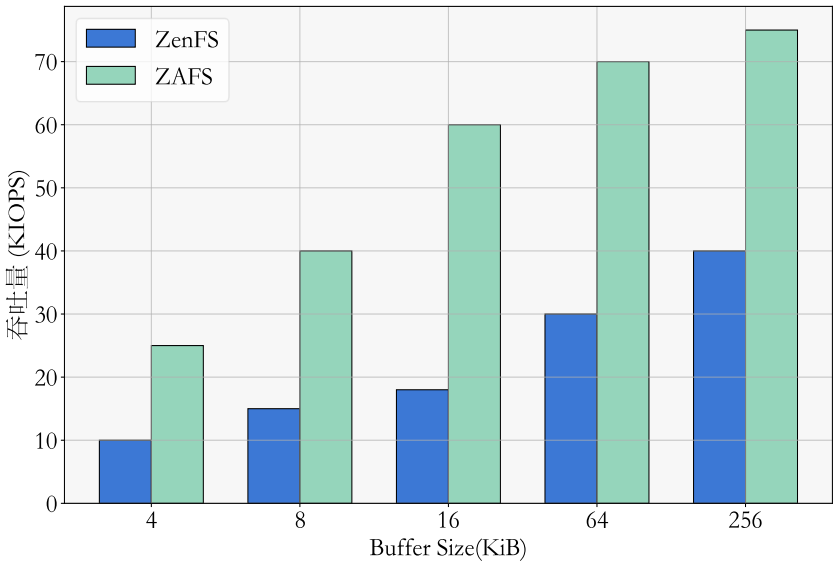

图 4-6 ZAFS 吞吐量测试

可以观察到，在所有评估的缓冲区大小上都优于 ZenFS 的 WALs，吞吐量提升了 1.9~3.5 倍。经测试，ZAFS 中的 Zone-Friends WAL 设计有效地缓解了高并发写入时的性能瓶颈，提升了吞吐量。

4.4.2 ZAFS 恢复时间测试

ZAFS 与 ZenFS 的恢复时间对比测试如图 4-7 所示。

ZAFS 的 WAL 设计使得 WAL 条目更大，引入了读开销，但是却比 ZenFS 所需的恢复时间更短。这是因为：首先，SSD 设备的读取速度很快，读取 WAL 文件膨胀部分所需的额外时间几乎可以忽略不记。另一方面，由于引入 SPDK 优化了 I/O 路径，相较于 ZenFS 对 Zone 的操作速度更快；并且在内存中保留了映射表，使得 WAL entry 的寻址更快。上述原因使得 ZAFS 的恢复时间反而短于 ZenFS。

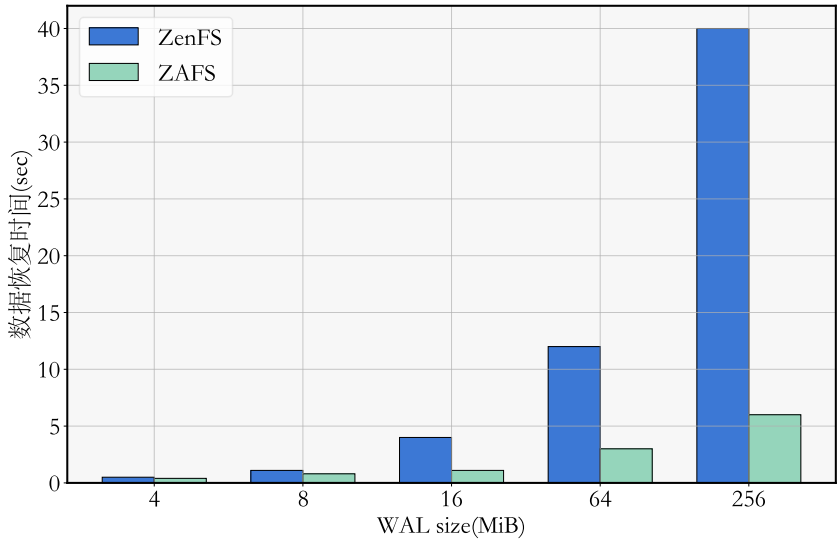

图 4-7 ZAFS 恢复时间测试

4.5 本章小结

本章对应于第三章中的内容顺序进行相应的测试实验。首先，通过模拟器和真实设备的对比，探讨了模拟器的优势和不足之处；然后，对 Inspur NS800G2-Z ZNS SSD 进行了基准测试，探讨了各个参数设置对于 ZNS 吞吐量的影响；最后通过写入和恢复实验，验证了 ZAFS 实现的有效性。

5 总结与展望

本课题对 Zoned Namespace (ZNS) 固态硬盘 (SSD) 进行了系统的探讨和分析。通过细致评估 ZNS SSD 的硬件特性，对 LinuxI/O 路径的分析，以及对 LSM-Tree 模型和 RocksDB 存储引擎的深入分析，发现现有工作未能充分发挥 ZNS SSD 的高性能和低延迟优势。基于此，本课题提出了一种 Zone- Append Write-Ahead Logging（WAL）写前日志方案，并实现 ZAFS（Zone-Append File System）文件系统设计，有效解决了基于 ZNS 的 RocksDB 在高写入环境下的性能瓶颈。

本文深入探讨了 ZNS SSD 的性能特性，包括硬件基准测试、RocksDB 性能评估和问题分析等。随后提出了 Zone-Aware File System（ZAFS）的设计与实现，并分析所提出的 A2-WAL 在提升系统性能方面的关键作用。在实验部分，通过模拟与实际设备的测试、FIO 基准测试分析了硬件本身的性能，通过基于 RocksDB 的 ZAFS 测试，验证了优化方案的有效性。硬件测试表明，块大小和多线程配置显著提升了 IOPS 和带宽，但随着块大小增加，多线程的性能提升效果减弱。应用测试表明，ZAFS 通过引入区追加命令和 A2-WAL 的设计，显著降低了写入延迟并减少了资源竞争，且 A2-WAL 没有增加崩溃时的恢复开销，其灾害恢复时间反而更短。

尽管取得了一些进展，本课题仍面临一些挑战。首先，ZNS SSD 的广泛应用需操作系统和应用软件的全面支持，涉及兼容性和技术适应性问题。其次，ZNS SSD 管理复杂性增加，需开发智能资源调度算法和系统监控工具。此外，还需针对特定应用场景继续优化性能策略，以最大化 ZNS SSD 的优势。

未来工作将聚焦优化 ZNS 的数据放置策略，并增强其在不同负载下的适应性和稳定性。同时，探索基于 ZNS SSD 的新型存储协议和文件系统，提升普适性。此外，还计划评估 ZNS SSD 在分布式存储系统中的应用潜力，探索其在大规模数据处理中的性能表现和成本效益。

致 谢

论文完成之际，回望过去的学习历程，感激之情油然而生。在这里，我要向所有支持与帮助过我的人表示衷心的感谢。

首先，我必须感谢我的导师，刘渝副研究员，感谢您在我的本科学习阶段给予的指导与关怀。刘老师不仅以其卓越的专业知识为我筑建了坚实的学术基础，更以其深沉的人文关怀启发我思考科学与生活的深远意义。您的悉心教导和严格要求，让我在遇到困惑时总能找到方向，是我人生旅程中的灯塔。

我还要特别感谢我的师兄，王鹏博士，他不仅在学术上给予我大量的帮助，更在日常生活中给予我关心和支持。王师兄对待科研的严谨态度和细致精神一直是我学习的榜样，他在实践中的指导极大地提高了我的工作效率。

此外，我还要感谢我们实验室的全体成员。是这个充满活力的团队为我提供了一个学习交流和科研探索的优越平台。每一次团队讨论都激发了我新的思考和灵感，每一次合作都让我深刻体会到团队合作的力量。

在我的求学之路上，每一步都凝聚了师长、师兄和同侪的智慧与努力。感谢你们的陪伴和支持，使我在这段珍贵的学习旅程中，不断前行，勇敢探索。

参考文献

[1] Rosenblum, M., Ousterhout, J.K. The design and implementation of a log-structured file system. ACM Transactions on Computer Systems (TOCS), 10(1), 1992, 26-52.

[2] Lee, S., Park, J., Kim, Y. et al. Whole-File-Content-based Prefetching for Read-Intensive Workloads in Colossus. Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST), 2015.

[3] Jannen, W., et al. BetrFS: A Right-Optimized Write-Optimized File System. Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST), 2015, 1-14. 301-313

[4] Yang, S., et al. Declustered RAID for Hot Storage with Dynamic Parity Allocation. Proceedings of the 11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), 2019, 7-12.

[5] C. Chakraborttii, H. Litz. Reducing write amplification in flash by death-time prediction of logical block addresses. HotStorage '21: 13th ACM Workshop on Hot Topics in Storage and File Systems, Virtual Event, USA, 27-28 Jul. 2021, ACM / USENIX Association, 2021: 1-12

[6] X. Hu, E. Eleftheriou, R. Haas, I. Iliadis, R. Pletka. Write amplification analysis in flash-based solid state drives. Proceedings of SYSTOR 2009: The Israeli Experimental Systems Conference 2009, Haifa, Israel, 4-6 May 2009, ACM, 2009: 1- 9

[7] Y. Li, P. Lee, J. Lui. Stochastic modeling of large-scale solid-state storage systems: Analysis, design tradeoffs and optimization. SIGMETRICS '13: ACM SIGMETRICS / International Conference on Measurement and Modeling of Computer Systems, Pittsburgh, PA, USA, 17-21 Jun. 2013, ACM, 2013: 179-190

[8] B. Houdt. A mean field model for a class of garbage collection algorithms in flash-based solid state drives. ACM SIGMETRICS Performance Evaluation Review, 2013, 41(1): 191-202

[9] J. Kang, J. Hyun, H. Maeng, S. Cho. The multi-streamed solid-state drive. 6th USENIX Workshop on Hot Topics in Storage and File Systems, HotStorage '14,

Philadelphia, PA, USA, 17-18 Jun. 2014, USENIX Association, 2014

[10] M. Bjørling, J. González, P. Bonnet. Lightnvm: The linux open-channel ssd subsystem. 15th USENIX Conference on File and Storage Technologies, FAST 2017, Santa Clara, CA, USA, 27 Feb. – 2 Mar. 2017, USENIX Association, 2017: 359-374

[11] Kang, Yangwook, Rekha Pitchumani, Pratik Mishra, Yang-suk Kee, Francisco Londono, Sangyoon Oh, Jongyeol Lee, and Daniel DG Lee. "Towards building a high-performance, scale-in key-value storage system." In Proceedings of the 12th ACM International Conference on Systems and Storage, pp. 144-154. 2019.

[12] Bjørling, M., Aghayev, A., Holmberg, H., Ramesh, A., Le Moal, D., Ganger, G. R., & Amvrosiadis, G. ZNS: Avoiding the Block Interface Tax for Flash-based SSDs. Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST 23), 2023.

[13] Han, K., Gwak, H., Shin, D., & Hwang, J. ZNS+: Advanced zoned namespace interface for supporting in-storage zone compaction. Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), 2021.

[14] Min, J., Zhao, C., Liu, M., & Krishnamurthy, A. eZNS: An elastic zoned namespace for commodity ZNS SSDs. Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23), 2023.

[15] 王鑫, 高志, & 张勇. (2020). 基于区域命名空间的固态硬盘垃圾回收算法. 中国计算机大会, 2020 (pp. 321-328). 北京: 中国计算机学会.

[16] Lee, C., Sim, D., Hwang, J., & Cho, S. (2015). F2FS: A new file system for 13th USENIX Conference on File and Storage Technologies (FAST 15)

[17] Cao, Z., Dong, S., Vemuri, S., & Du, D. H.C. (2020). Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook. In 18th USENIX Conference on File and Storage Technologies (FAST '20), Santa Clara, CA, USA: USENIX Association, pp. 209-223.

[18] 刘扬, 金培权.. ZB+-tree：一种 ZNS SSD 感知的新型索引结构. 计算机研究与发展, 59(1), 2022，102-115.

[19] 阿里巴巴集团. 一种用于区域命名空间固态硬盘的数据分布方法. 中国. 专

利号: CN107845921A. 2017.

[20] Bergman, S., Cassel, N., Bjørling, M., et al. ZNSwap: Un-block your swap. ACM Transactions on Storage (TOS), 1-20.

[21] O'Neil, P., Cheng, E., Gawlick, D., & O'Neil, E. The log-structured merge-tree (LSM-tree). Acta Informatica, 1-26

[22] Hewitt, J., & Sivasubramanian, S. . Cassandra: A decentralized structured storage system. Proceedings of the 3rd ACM SIGOPS/EuroSys European Conference on Computer Systems (EuroSys '10), 2010，1(1), 177–190.

[23] Kim, H., Shim, Y., Park, H., et al. . NVMeVirt: Towards a Scalable and Efficient NVMe-oF SSD Virtualization. Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST 23),2023， 1(1), 111–124.

[24] Taft, R., Sharif, I., Matei, A., et al.. CockroachDB: The Resilient Geo-Distributed SQL Database. Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20), (2020)，1(1), 1203–1216.

[25] Huang, D., Liu, Q., Cui, Q., et al.. TiDB: A Raft-based HTAP Database. Proceedings of the VLDB Endowment, 13(12),2020， 3072–3084.